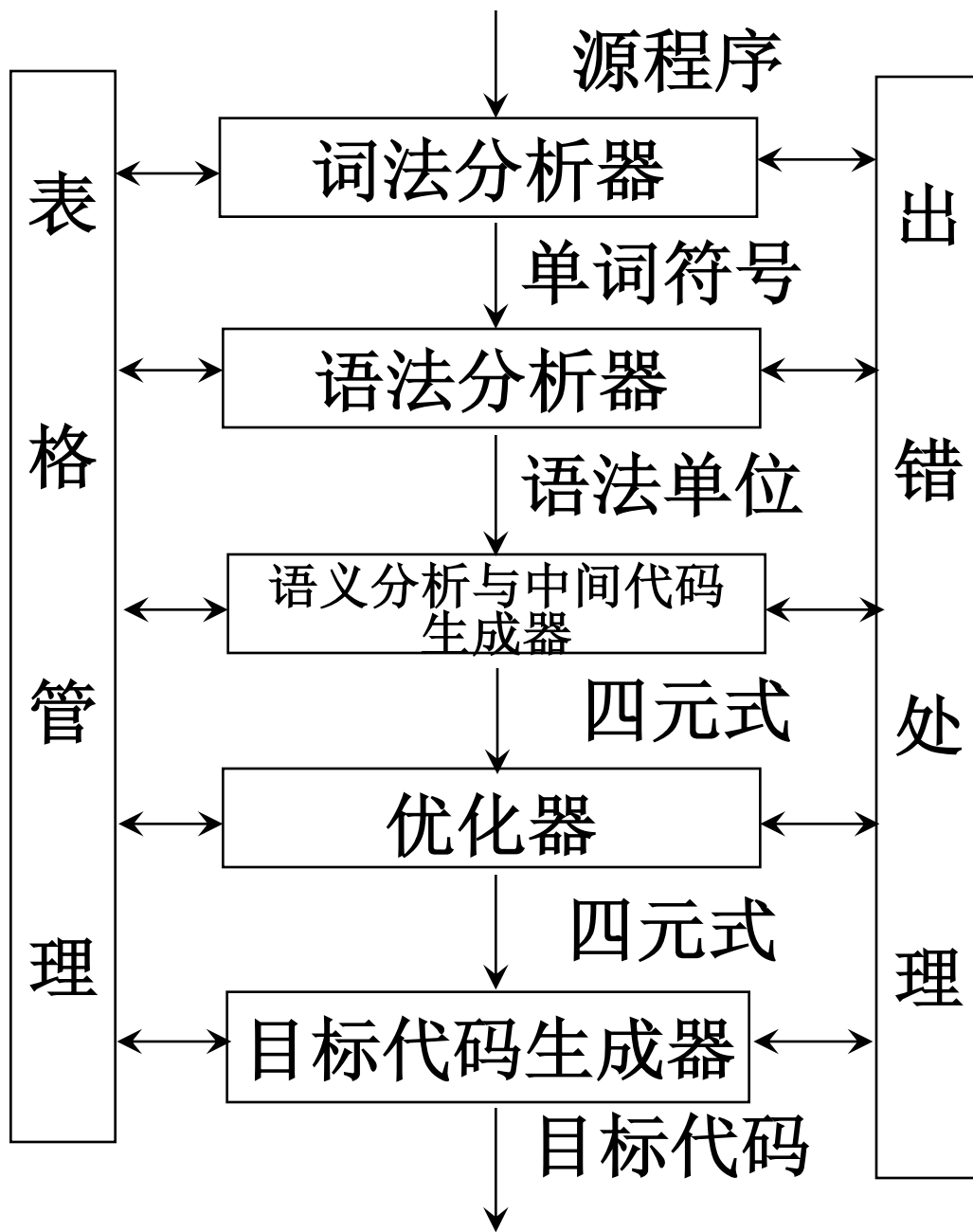


第三章

词法分析



第三章 词法分析

- 词法分析的任务：从左至右逐个字符地对源程序进行扫描，产生一个个单词符号，把作为字符串的源程序改造成成为单词符号串的中间程序。
- 词法分析器(Lexical Analyzer) 又称扫描器(Scanner)：执行词法分析的程序

第三章 词法分析

- 3.1 对于词法分析器的要求
- 3.2 词法分析器的设计
- 3.3 正规表达式与有限自动机
- 3.4 词法分析器的自动产生

3.1.1 词法分析器的功能和输出形式

- 功能:输入源程序, 输出单词符号
- 单词符号: 程序语言的基本语法符号
- 单词符号的种类:
 - 关键字: 如 if, while, ...
 - 标识符: 表示各种名字: 如变量名、数组名和过程名等
 - 常数: 如整型、实型、布尔型、文字型等
 - 运算符: +, -, *, /, ...
 - 界符: 逗号、分号、括号等

- 输出的单词符号的表示形式：
(单词种别, 单词符号的属性值)
- 单词种别通常用整数编码表示。
 - 若一个种别只有一个单词符号, 则种别编码就代表该单词符号。假定关键字、运算符和界符都是一符一种。
 - 若一个种别有多个单词符号, 则对于每个单词符号, 给出种别编码和属性值。
 - 标识符单列一种; 将存放它的有关信息的符号表项的指针作为其属性值。
 - 常数按类型分种; 存放它的常数表项的指针作为其属性值。

单词符号种别编码

界符 一符一种	种别编码	属性值
(	81	-
)	82	-
;	83	-
{	84	-
.....		

运算符 一符一种	种别编码	属性值
+	41	-
>=	48	-
--	57	-
.....		

常数 一型一种	种别编码	属性值
整型常数	100	指针值
.....		

标识符	种别编码	属性值
标识符	111	指针值

关键字 一字一种	种别编码	属性值
else	15	-
if	17	-
while	20	-
.....		

例 C程序

- `while (i>=50) j--;`

- 输出单词符号:

< 20, - >

< 81, - >

< 111, 指向i的符号表项的指针 >

< 48, - >

< 100, 指向50的常数表项的指针 >

< 82, - >

< 111, 指向i的符号表项的指针 >

< 57, - >

< 83, - >

例 C程序

■ while (i>=50) j--;

■ 输出单词符号:

< while, - >

< (, - >

< id, 指向i的符号表项的指针 >

< >=, - >

< integer, 指向50的常数表项的指针 >

<), - >

< id, 指向i的符号表项的指针 >

< --, - >

< ;, - >

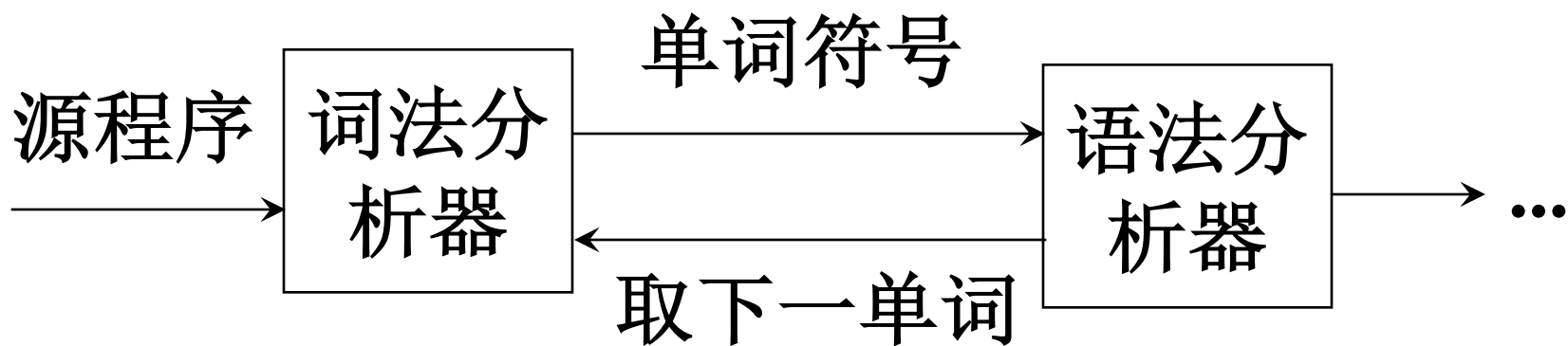
3.1.2 词法分析器作为一个独立子程序

■ 把词法分析器安排成一个子程序

每当语法分析器需要一个单词符号时就调用这个子程序。

每一次调用，词法分析器就从输入串中识别出一个单词符号，把它交给语法分析器。

词法分析器



3.2 词法分析器的设计

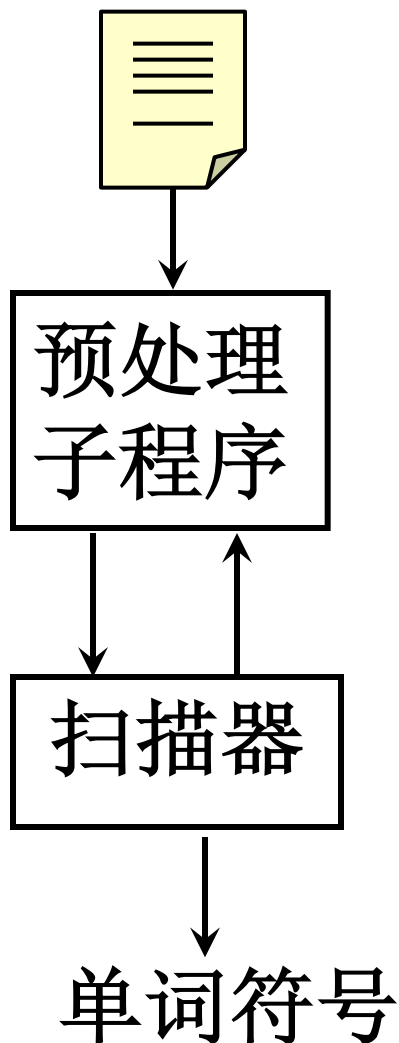


图3.1 词法分析器

3.2.1 输入、预处理

- 输入串放在输入缓冲区中。
- 预处理子程序：剔除无用的空白、跳格、回车和换行等编辑性字符和注释。

3.2.3 状态转换图

- 使用状态转换图是设计词法分析程序的一种好途径。
- 转换图是一张有限方向图。
 - 结点代表状态，用圆圈表示。
 - 状态之间用箭弧连结，箭弧上的标记(字符)代表在射出结点状态下可能出现的输入字符或字符类。
 - 一张转换图只包含有限个状态，其中有一个为初态，至少要有一个终态(用双圈表示)。

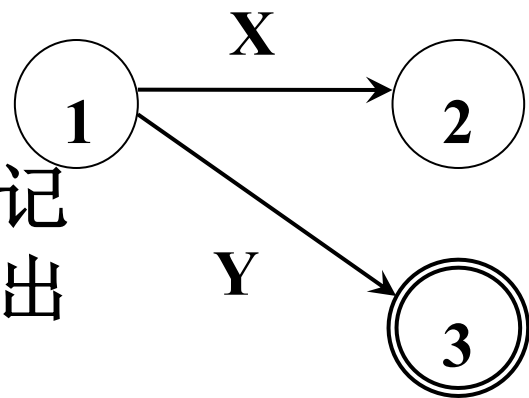


图3.2(a) 转换图示例

- 一个状态转换图可用于识别(或接受)一定的字符串。

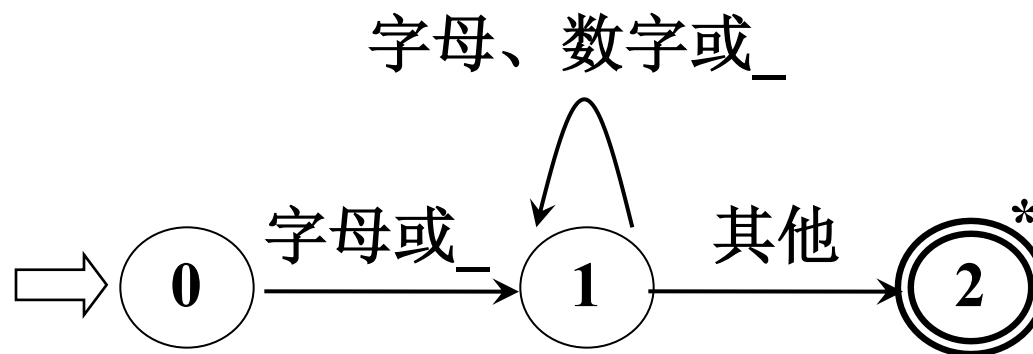


图3.2(b) 识别标识符的状态转换图

- 终态结上星号*: 意味着多读进了一个不属于标识符部分的字符, 应把它退还给输入串。

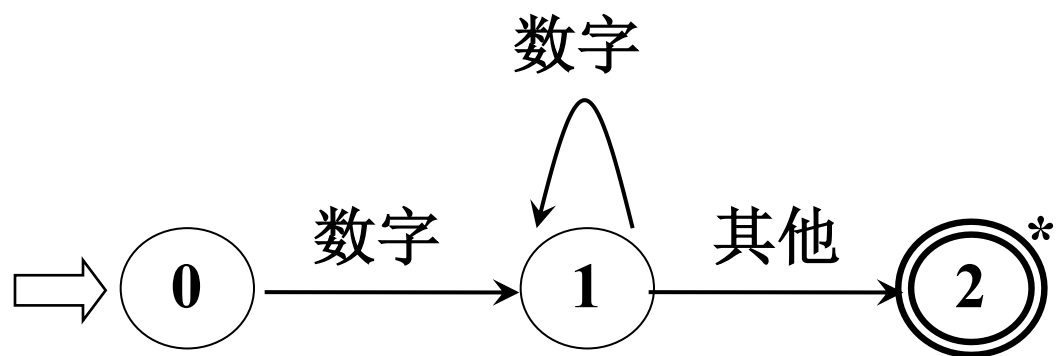
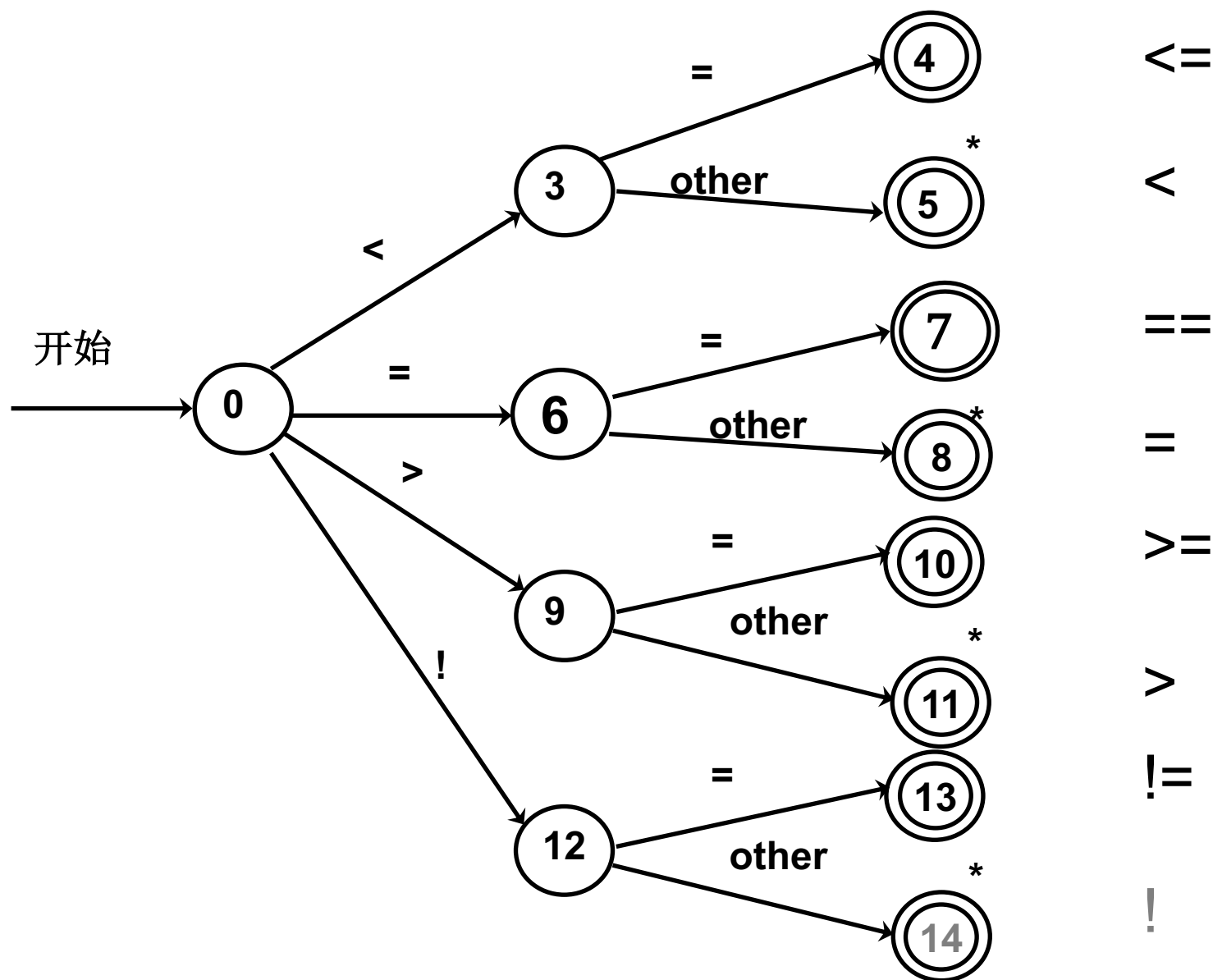
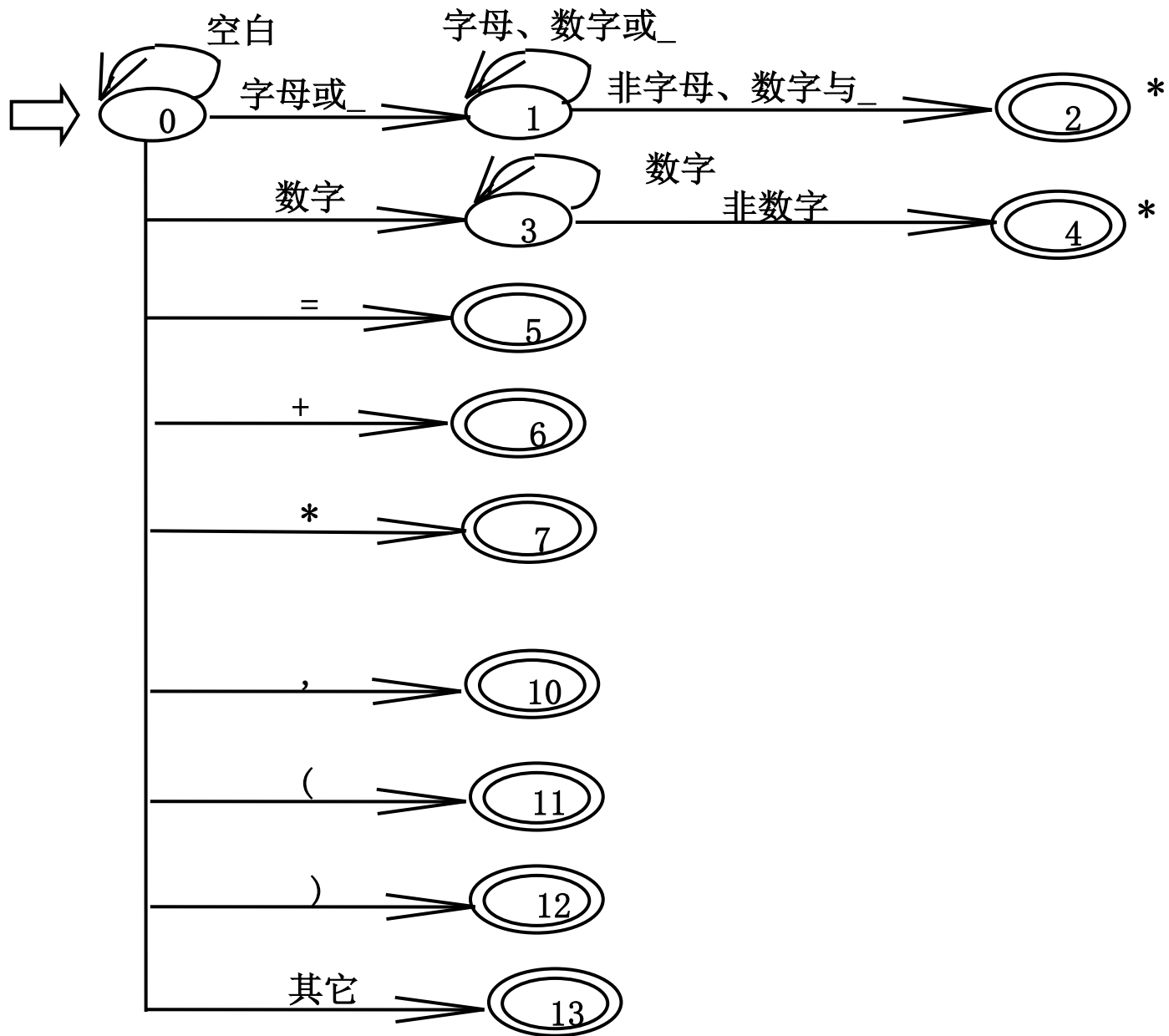


图3.2(c) 识别整数的状态转换图

■ 关系运算符的转换图





3.2.4 状态转换图的实现

- 每个状态结点对应一小段程序。

■ 全局变量与过程

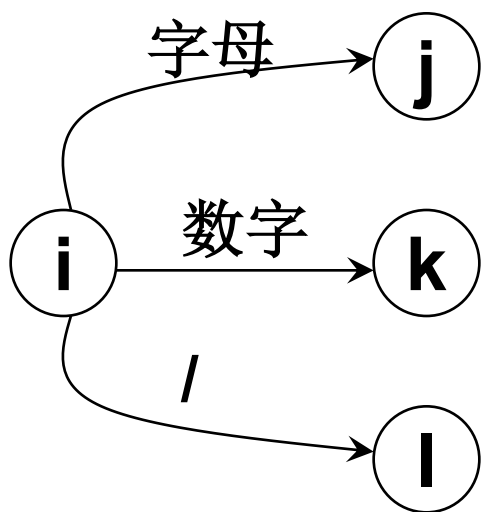
- 1) **ch** 字符变量，存放最新读进的源程序字符
- 2) **strToken** 字符数组，存放构成单词符号的字符串
- 3) **GetChar** 子程序过程，将下一输入字符读到**ch**中，搜索指示器前移一字符位置
- 4) **GetBC** 子程序过程，检查**ch**中的字符是否为空白符，若是，则调用**GetChar**直至**ch**中读入一个非空白字符
- 5) **Concat** 子程序，将**ch**中的字符连接到**strToken**

- 6) **IsLetter**和 **IsDisgital** 布尔函数过程，分别判断**ch**中字符是否为字母和数字
IsUnderscore 布尔函数过程，判断**ch**中字符是否为下划线
- 7) **Reserve** 整型函数过程，对**strToken**中的字符串查找保留字表，若它是一个保留字则返回它的编码，否则返回0
- 8) **Retract** 子程序过程，把搜索指示器回调一个字符位置，将**ch**置为空白字符
- 9) **InsertId** 指针函数过程，将**strToken**中的标识符插入符号表，返回符号表指针
- 10) **InsertConst** 指针函数过程，将**strToken**中的常数插入常数表，返回常数表指针。

3.2.4 状态转换图的实现

- 思想：每个状态结点对应一小段程序。
- 做法：

1) 不含回路的分叉结点：可对应一个SWITCH 语句或一组IF-THEN-ELSE语句

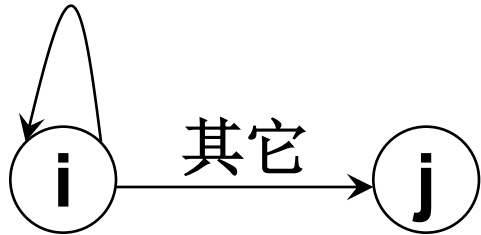


```
GetChar();  
if (IsLetter( )) {...状态j的对应程序段...;}  
else if (IsDigit( )) {...状态k的对应程序段...;}  
else if (ch='/') {...状态l的对应程序段...;}  
else {...错误处理...;}
```

图3.4(a) 具有不含回路的分叉结点的转换图

2) 含回路的状态结点：可对应一段由WHILE语句和IF语句构成的程序段。

字母或数字



```
GetChar( );  
while (IsLetter( ) or IsDigit( ))  
    GetChar( );  
...状态j的对应程序段...
```

图3.4(b) 具有含回路的状态结点的转换图

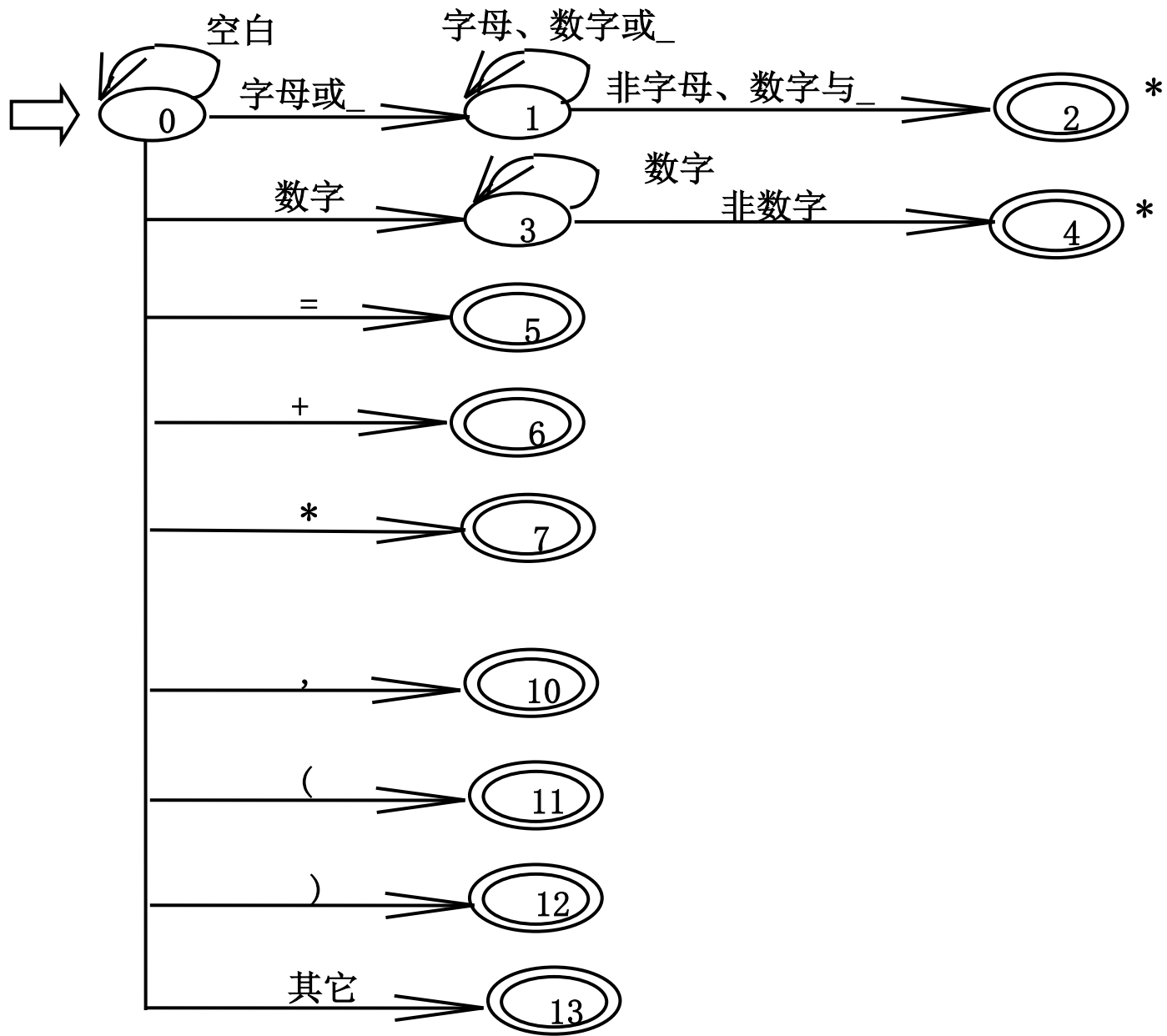
3)终态结点：对应语句为

RETURN (code, value)

其中，**code**为单词种别编码，**value**或是单词符号的属性值，或无定义。

意味着从分析器返回到调用者，一般指返回到语法分析器。

带星号*的终态结点：多读进了一个不属于现行单词符号的字符，应予退回，即搜索指示器回调一字符位置。



单词符号	种别编码	助忆符	内码值
DIM	1	\$DIM	—
IF	2	\$IF	—
DO	3	\$DO	—
STOP	4	\$STOP	—
END	5	\$END	—
标识符	6	\$ID	内部字符串 标准二进制形式
常数(数)	7	\$INT	
=	8	\$ASSIGN	—
+	9	\$PLUS	—
*	10	\$STAR	—
**	11	\$POWER	—
,	12	\$COMMA	—
(	13	\$LPAR	—
)	14	\$RPAR	—

```
if (IsLetter() or IsUnderline() ) ... ;  
else if (IsDigit()) ... ;  
else if (ch = '=' ) ... ;  
else if (ch = '+' ) ... ;  
else if (ch = '*' ) ... ;  
  
else if (ch = ';' ) ... ;  
else if (ch = '(' ) ... ;  
else if (ch = ')') ... ;  
  
else ... ;
```

```

int code;
struct SymbolTable *value;    /*符号表指针*/
strToken := “ ”;    /*置strToken为空串*/
GetChar(); GetBC();
if (IsLetter() or IsUnderline() )
begin
    while( IsLetter() or IsDigit() or IsUnderscore() )
    begin
        Concat(); GetChar();
    end
    Retract();
    code := Reserve(); /*查询保留字表*/
    if (code = 0)
    begin
        value := InsertId(strToken);
        return ($ID, value);
    end
    else
        return (code, -);
end
end

```

```
else if (IsDigit())
begin
    while (IsDigit())
    begin
        Concat( ); GetChar( );
    end
    Retract();
    value := InsertConst(strToken);
    return($INT, value);
end
else if (ch = '=') return ($ASSIGN, -);
else if (ch = '+') return ($PLUS, -);
```

else if (ch = '*') return (\$STAR, -);

else if (ch = ';') return (\$SEMICOLON, -);

else if (ch = '(') return (\$LPAR, -);

else if (ch = ')') return (\$RPAR, -);

else ProcError(); /* 错误处理*/

3.3 正规表达式与有限自动机

- 3.3.1 正规式与正规集
- 3.3.2 确定有限自动机（DFA）
- 3.3.3 非确定有限自动机（NFA）
- 3.3.4 正规文法与有限自动机的等价性
- 3.3.5 正规式与有限自动机的等价性
- 3.3.6 确定有限自动机的化简

3.3.1 正规式与正规集

- 正规式 (Regular Expressions): 描述语言
 - 正规表达式可以描述所有通过对某个字母表上的符号应用并、连接和闭包运算而得到的语言。
- 正规集: 正规式定义的语言

■ 正规式可以由较小的正规式递归地构建。

给定字母表 Σ :

- 1) ε 和 Φ 是 Σ 上的正规式, 所表示的正规集分别为 $\{\varepsilon\}$ 和 Φ ;
- 2) 任何 $a \in \Sigma$, a 是 Σ 上的正规式, 所表示的正规集为 $\{a\}$;
- 3) 设 r 和 s 是 Σ 上的正规式, 所表示的正规集为 $L(r)$ 和 $L(s)$, 则

$r|s$ 是 Σ 上的正规式, 所表示的正规集 $L(r) \cup L(s)$;

rs 是 Σ 上的正规式, 所表示的正规集 $L(r)L(s)$;

r^* 是 Σ 上的正规式, 所表示的正规集 $(L(r))^*$;

正规式运算符的优先顺序:

先闭包 $*$, 后连接 $\cdot$, 最后或 $|$ 。

■ 例3.a

令 $\Sigma=\{a,b\}$ ， Σ 上的正规式和相应的正规集：

正规式

正规集

$a|b$

$\{a, b\}$

$(a|b)(a|b)$

$\{aa, ab, ba, bb\}$

长度为2的所有串的集合

a^*

$\{\varepsilon, a, aa, aaa, \dots\}$

零个或多个a组成的串的集合

$(a|b)^*$

$\{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$

零个或多个a或b组成的串的集合

■ 例3.1

令 $\Sigma = \{a, b\}$, Σ 上的正规式和相应的正规集:

正规式	正规集
-----	-----

ba^*	Σ 上所有以 b 为首后跟任意多个 a 的字的集合
--------	--------------------------------------

$a(a b)^*$	Σ 上所有以 a 为首的字的集合
------------	---------------------------

$(a b)^*(aa bb)(a b)^*$	Σ 上所有含有两个相继的 a 或两个相继的 b 的字的集合
-------------------------	--

■ 例3.2

令 $\Sigma = \{A, B, 0, 1\}$, 则:

正规式

$(A|B)(A|B|0|1)^*$

$(0|1)(0|1)^*$

正规集

Σ 上的“标识符”的全体

Σ 上的“数”的全体

3.3.2 确定有限自动机(DFA)

- 有限自动机分为两类:
 - 确定有限自动机(DFA): 对每个状态及自动机输入表中的每个符号, 有且只有一条离开该状态、以该符号为标号的边。
 - 非确定有限自动机(NFA): 对其边上的标号没有任何限制, 同一符号可以有 multiple 条边离开同一状态, 并且空串 ϵ 也可以作为标号。

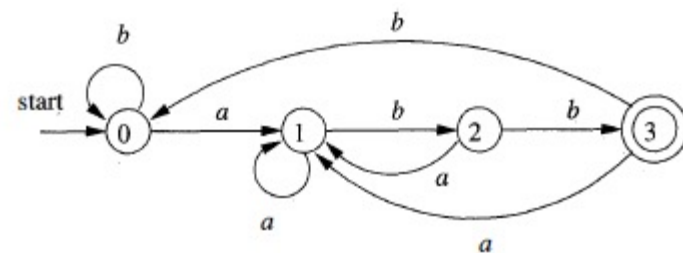


Figure 3.28: DFA accepting $(a|b)^*abb$

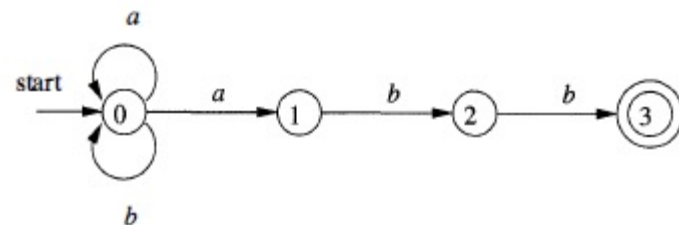


Figure 3.24: A nondeterministic finite automaton

$(a|b)^*abb$

3.3.2 确定有限自动机(DFA)

确定有限自动机 (DFA) M 是一个五元式

$$M=(S, \Sigma, \delta, s_0, F),$$

其中：

S : 有穷状态集,

Σ :有穷输入字母表,

δ :从 $S \times \Sigma$ 至 S 的单值部分映射, $\delta(s, a)=s'$ 表示:
当现行状态为 s , 输入字符为 a 时, 将转换到下一状态 s' 。我们称 s' 为 s 的一个后继状态。

$s_0 \in S$ 是唯一的初态;

$F \subseteq S$: 终态集。

- 例如: DFA $M = (\{0, 1, 2, 3\}, \{a, b\}, \delta, 0, \{3\})$, 其中 δ 为:

$$\delta(0, a) = 1$$

$$\delta(0, b) = 2$$

$$\delta(1, a) = 3$$

$$\delta(1, b) = 2$$

$$\delta(2, a) = 1$$

$$\delta(2, b) = 3$$

$$\delta(3, a) = 3$$

$$\delta(3, b) = 3$$

	a	b
0	1	2
1	3	2
2	1	3
3	3	3

状态转换矩阵

- DFA可表示成（确定的）状态转换图。假定DFA M 含有 m 个状态和 n 个输入字符，那么，这个图含有 m 个状态结点，每个结点顶多含有 n 条箭弧射出，每条箭弧用 Σ 中的一个不同输入字符来作标记，整张图含有唯一的一个初态结点和若干个终态结点。

	a	b
0	1	2
1	3	2
2	1	3
3	3	3

状态转换矩阵

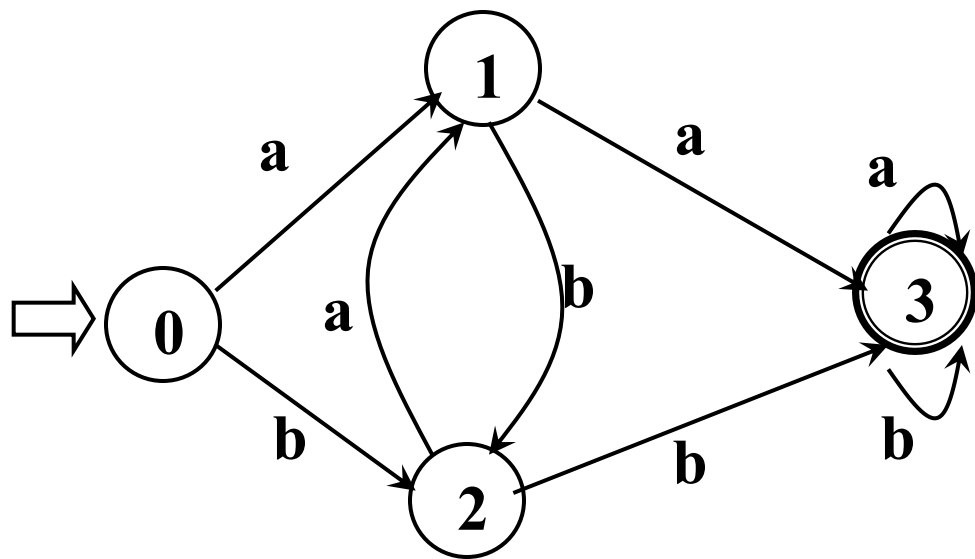
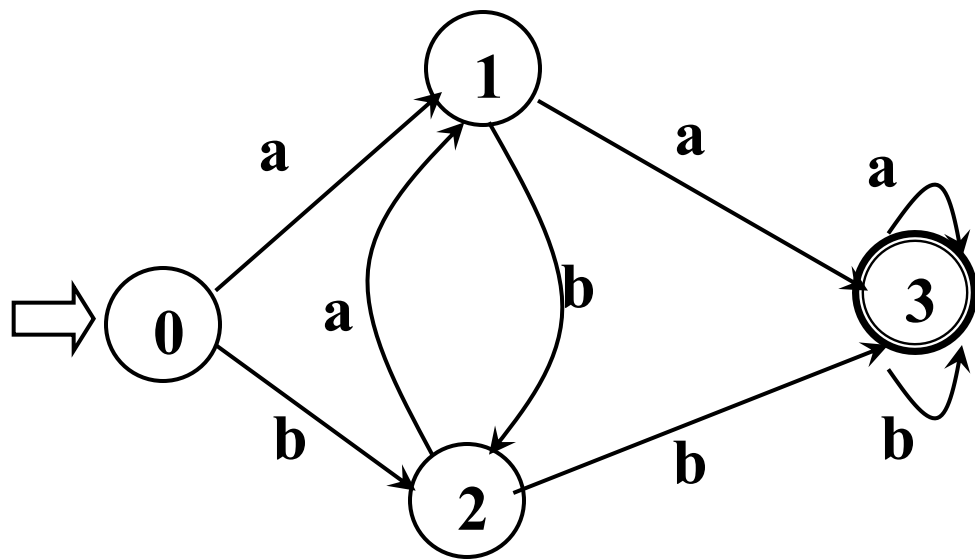


图3.5 状态转换图

- 对于 Σ^* 中的任何字 α ，若存在一条从初态结点到某一终态的通路，且这条通路上所有弧的标记符连接成的字等于 α ，则称 α 为DFA M 所识别(读出或接收)
- DFA M 所能识别的字的全体记为 $L(M)$ 。



能识别 Σ 上所有含有相继两个a或相继两个b的字

图3.5 状态转换图

3.3.3 非确定有限自动机(NFA)

- 定义：一个非确定有限自动机(NFA) M 是一个五元式 $M = (S, \Sigma, \delta, S_0, F)$ ，其中：

S : 有穷状态集；

Σ : 有穷输入字母表；

δ : 状态转换函数，从 $S \times \Sigma^*$ 到 S 的子集的映射；

$S_0 \subseteq S$: 非空初态集；

$F \subseteq S$: 终态集。

- 对于 Σ^* 中的任何一个字 α ，若存在一条从某一初态结点到某一终态结点的通路，且这条通路上所有弧的标记字依序连接成的字（忽略那些标记为 ϵ 的弧）等于 α ，则称 α 为NFA M所识别(读出或接收)

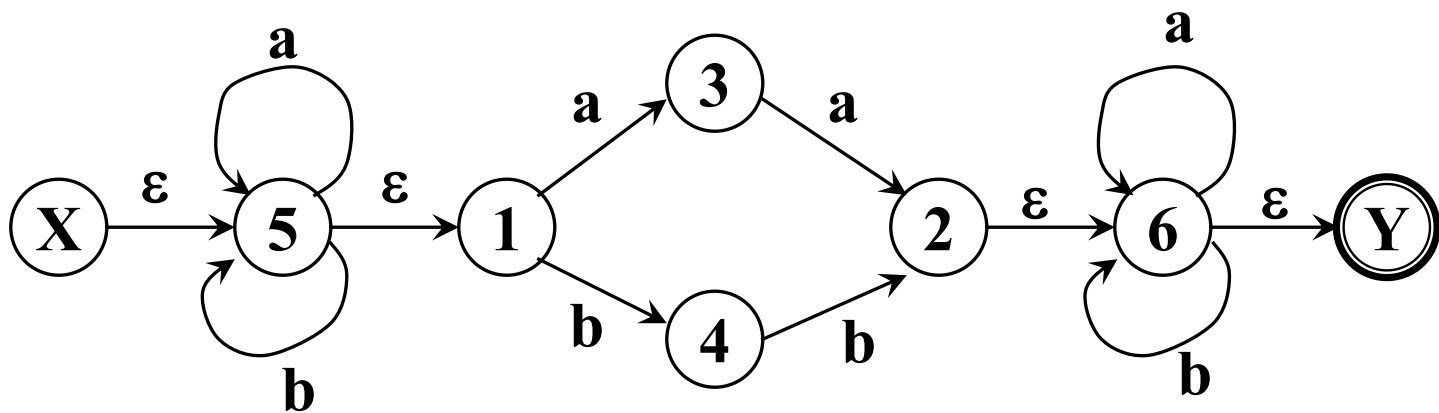


图3.6 非确定有限自动机

- 定义：对于任何两个有限自动机 M 和 M' ，如果 $L(M)=L(M')$ ，则称 M 与 M' 等价。
- 对于每个NFA M 存在一个DFA M'' ，使得 $L(M)=L(M'')$ 。

证明:

1. 假定NFA $M = \langle S, \Sigma, \delta, S_0, F \rangle$, 对M的状态转换图进行以下改造:
 - 1) 引进新的初态结点X和终态结点Y, $X, Y \notin S$,
从X到 S_0 中任意状态结点连一条 ϵ 箭弧, 从F中任意状态结点连一条 ϵ 箭弧到Y。
 - 2) 对M的状态转换图进一步施行替换, 其中k是新引入的状态。

按下面的三条规则对V进行分裂:

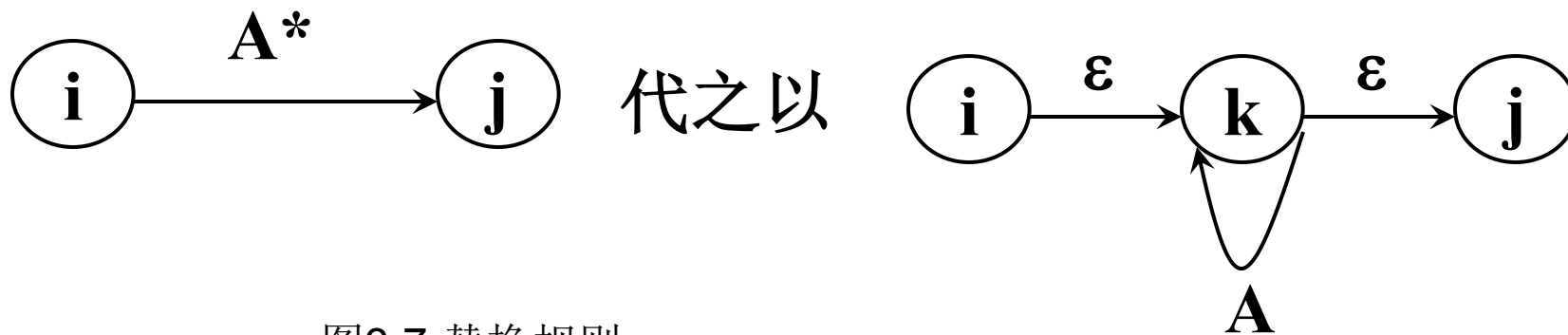


图3.7 替换规则

- 重复这种过程直至状态转换图中的每条箭弧上的标记或为 ϵ ，或为 Σ 中的单个字符，最终得到一个NFA M' ，显然 $L(M')=L(M)$

- 识别所有含相继两个a或相继两个b的字

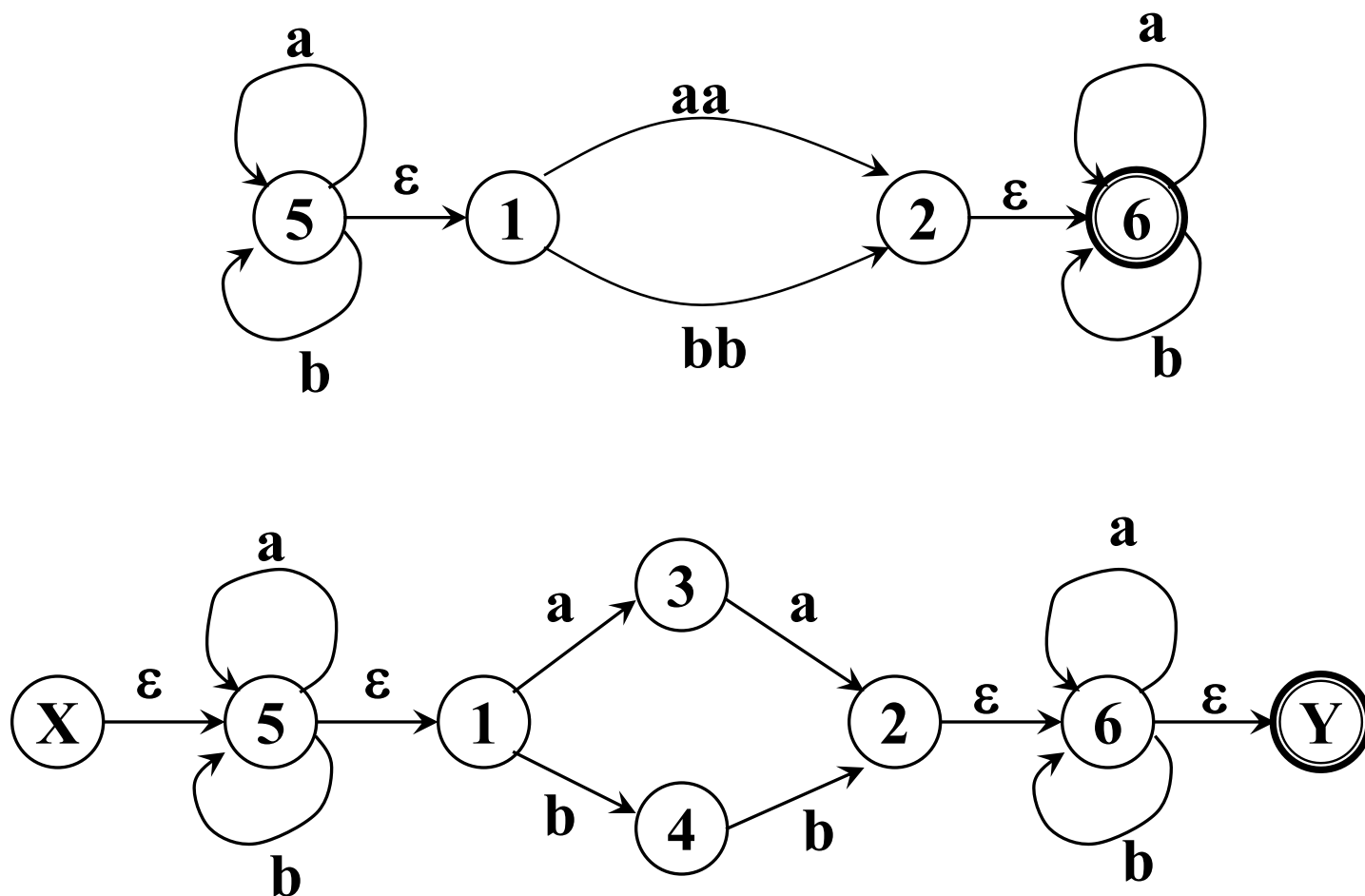


图3.6 非确定有限自动机 $(a|b)^*(aa|bb)(a|b)^*$

2. 将NFA M' 变换为DFA—子集构造法

- 在NFA状态上的运算

□状态 s 的 ϵ -闭包，表示为 $\epsilon\text{-closure}(s)$ ，定义为一状态集，从 s 出发经过任意条 ϵ 弧而能到达的状态集合。

例：

$$\epsilon\text{-closure}(X) = \{X, 5, 1\}$$

$$\epsilon\text{-closure}(5) = \{5, 1\}$$

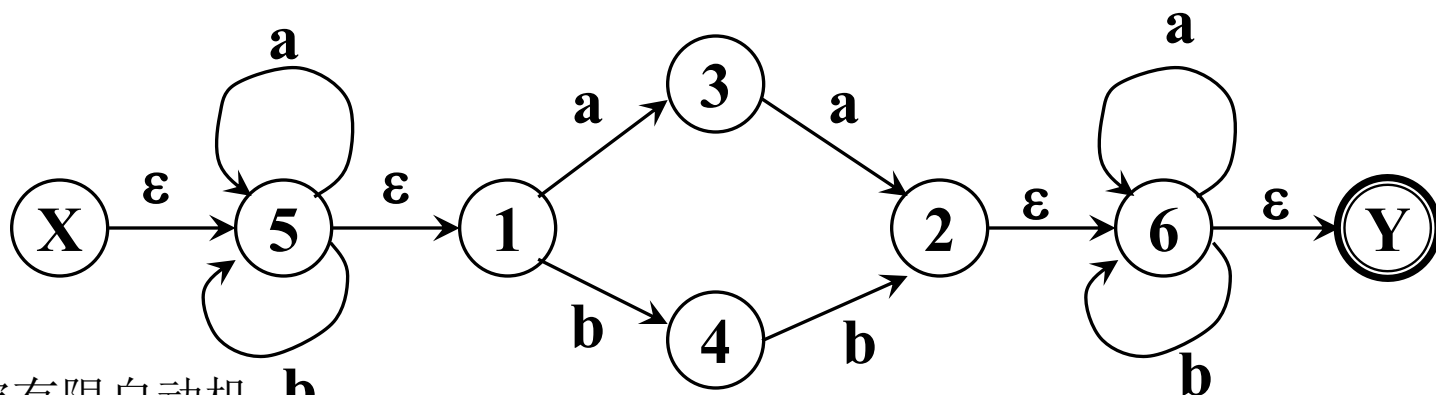


图3.6 非确定有限自动机

2. 将NFA M' 变换为DFA—子集构造法

- 在NFA状态上的运算

□ 状态集合 I 的 ϵ -闭包，表示为 **ϵ -closure(I)**，定义为一状态集，从集合 I 的任意状态 s 出发经过任意条 ϵ 弧而能到达的状态集合。

例：

设 $I = \{X, 5\}$ ，则

ϵ -closure(I) = $\{X, 5, 1\}$

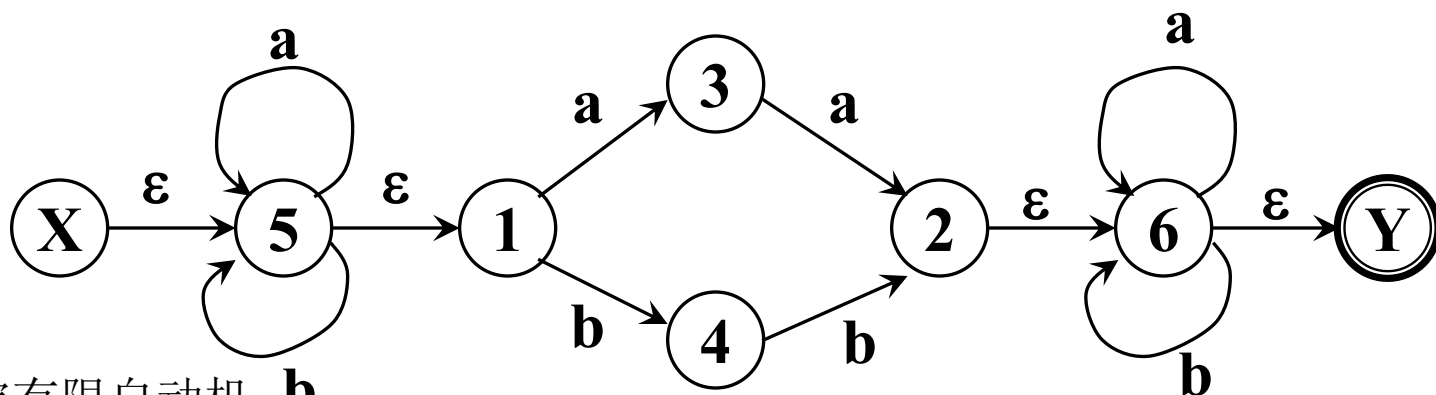


图3.6 非确定有限自动机

2. 将NFA M' 变换为DFA—子集构造法

- 在NFA状态上的运算

□状态集合 I 的 a 弧转换, 表示为 $\text{move}(I, a)$, 定义为一状态集 J , 从集合 I 中的某一状态结点出发经过一条 a 弧而到达的状态结点的全体。

例: 设 $I=\{X, 5, 1\}$, 则
 $J=\text{move}(I, a) = \{5, 3\}$

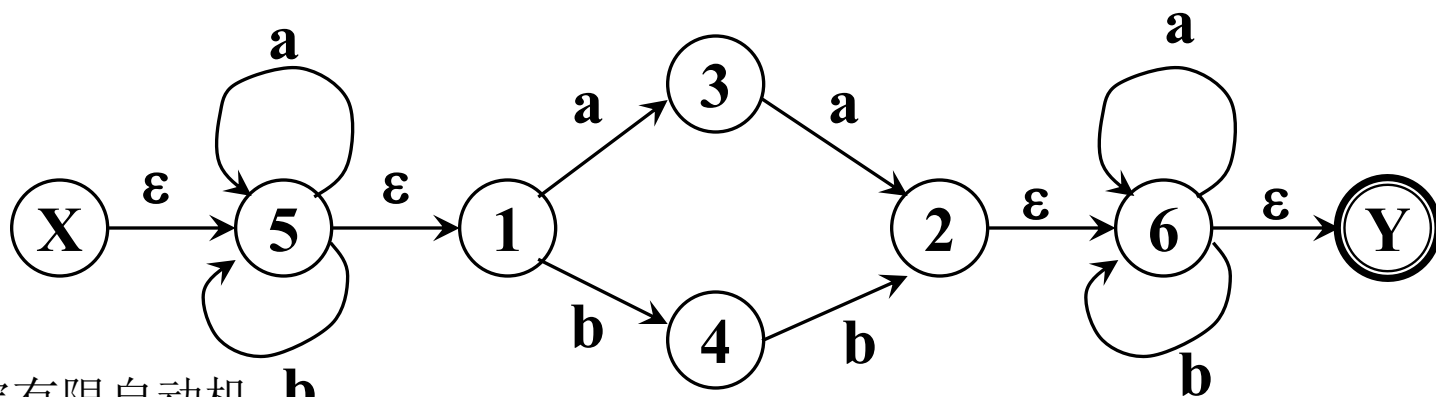


图3.6 非确定有限自动机

2. 将NFA M' 变换为DFA—子集构造法

- 在NFA状态上的运算

□ 定义 $I_a = \varepsilon\text{-closure}(J)$,
其中, $J = \text{move}(I, a)$ 。

例: 设 $I = \{X, 5, 1\}$, 则
 $J = \text{move}(I, a) = \{5, 3\}$, 则
 $I_a = \varepsilon\text{-closure}(J) = \{5, 1, 3\}$

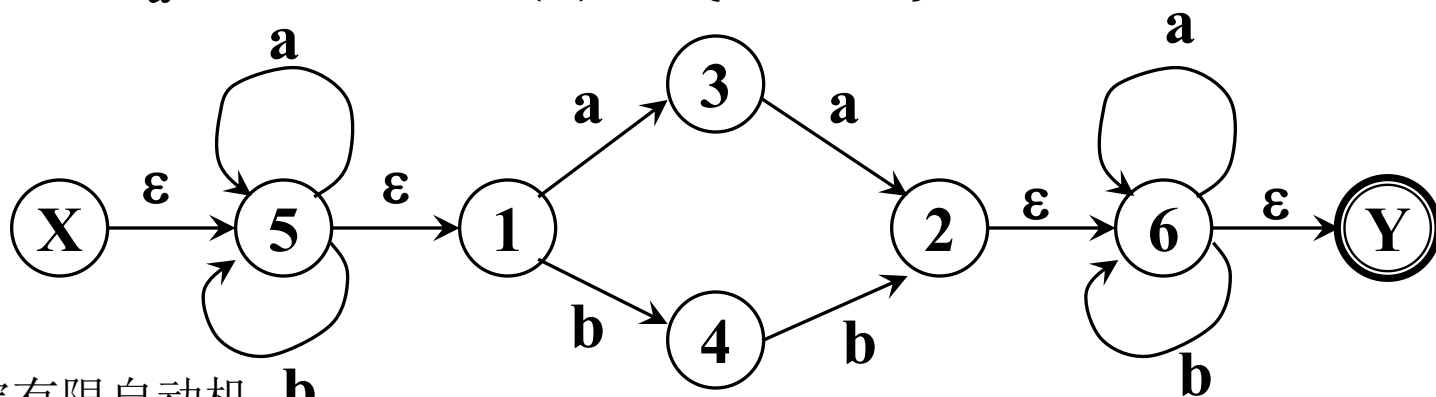


图3.6 非确定有限自动机

2. 将NFA M' 变换为DFA—子集构造法

- 在NFA状态上的运算

□ 定义 $I_a = \varepsilon\text{-closure}(J)$,
其中, $J = \text{move}(I, a)$ 。

例: 设 $I = \{X, 5, 1\}$, 则
 $J = \text{move}(I, b) = \{5, 4\}$, 则
 $I_b = \varepsilon\text{-closure}(J) = \{5, 1, 4\}$

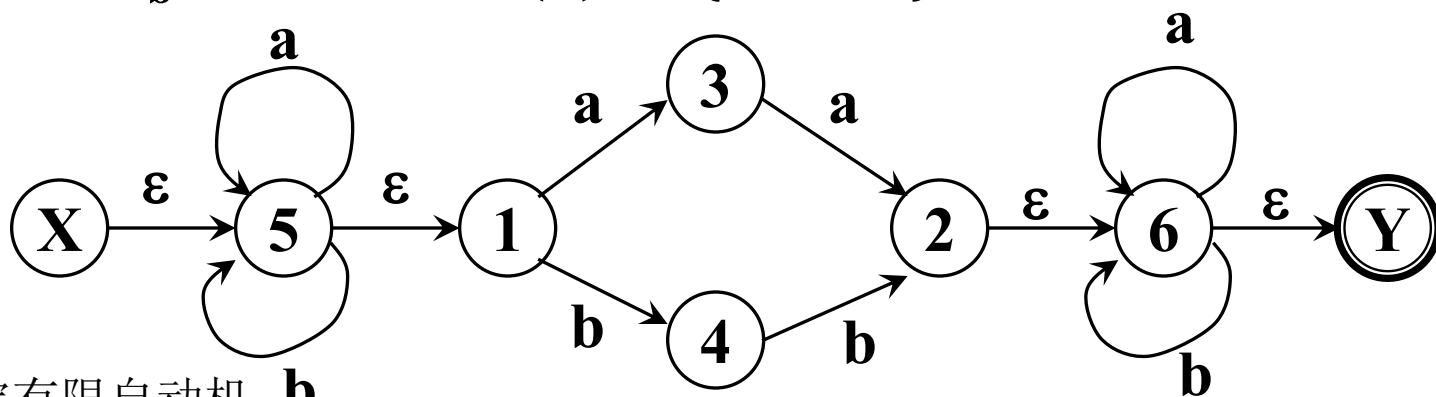


图3.6 非确定有限自动机

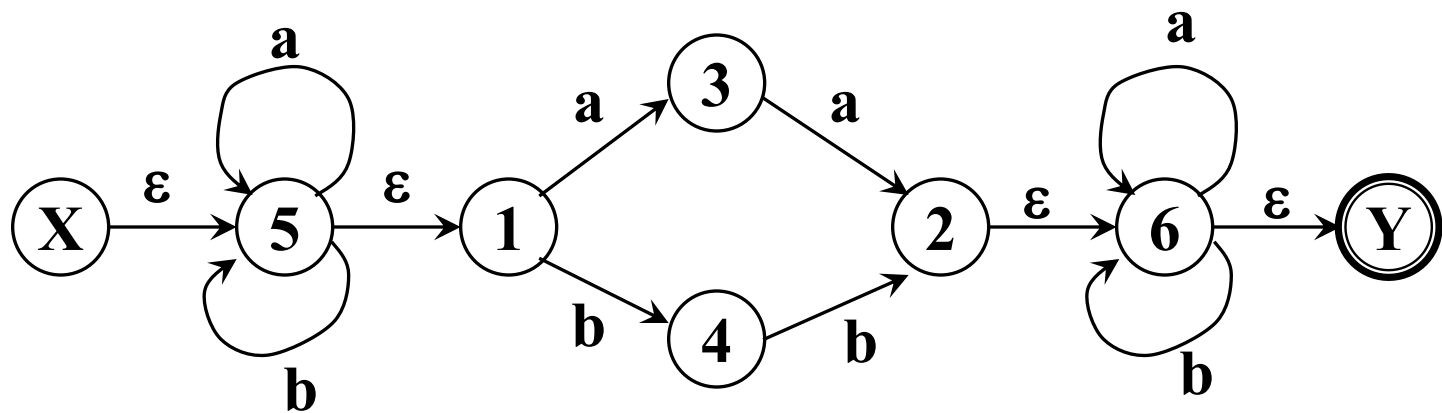
■ 确定化的过程:

假定 $\Sigma = \{a_1, \dots, a_k\}$,

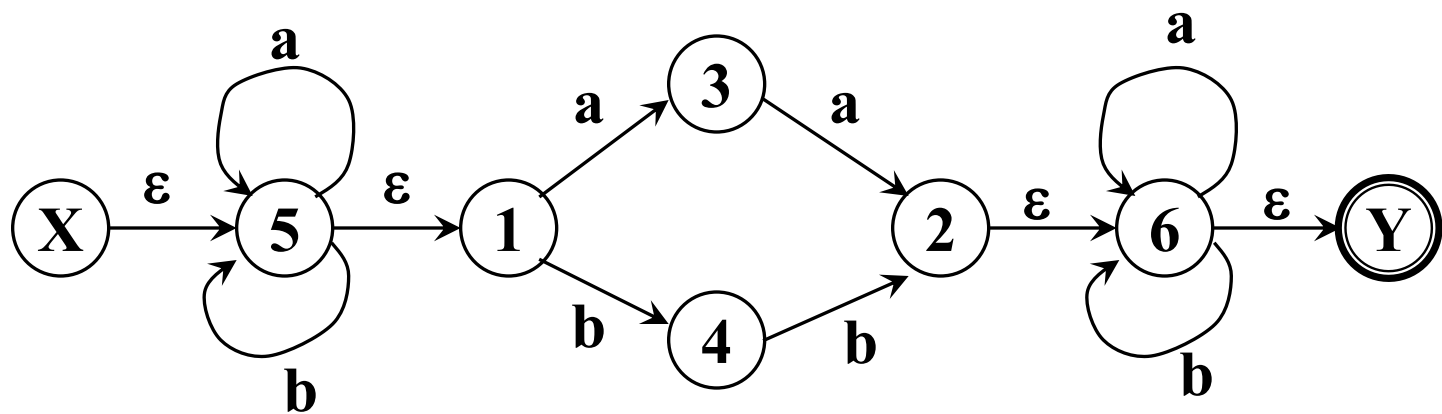
构造一张表, 表的每一行含有 $k+1$ 列:

首先，置第1行第1列为 ϵ -closure($\{X\}$),

I	I_a	I_b
ϵ-closure($\{X\}$)		

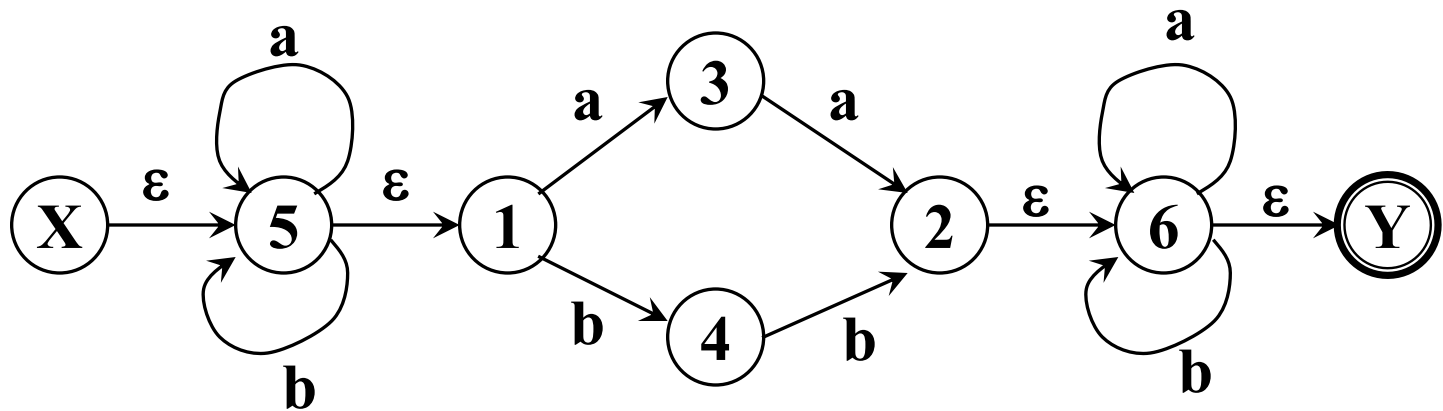


I	I_a	I_b
{X,5,1}		

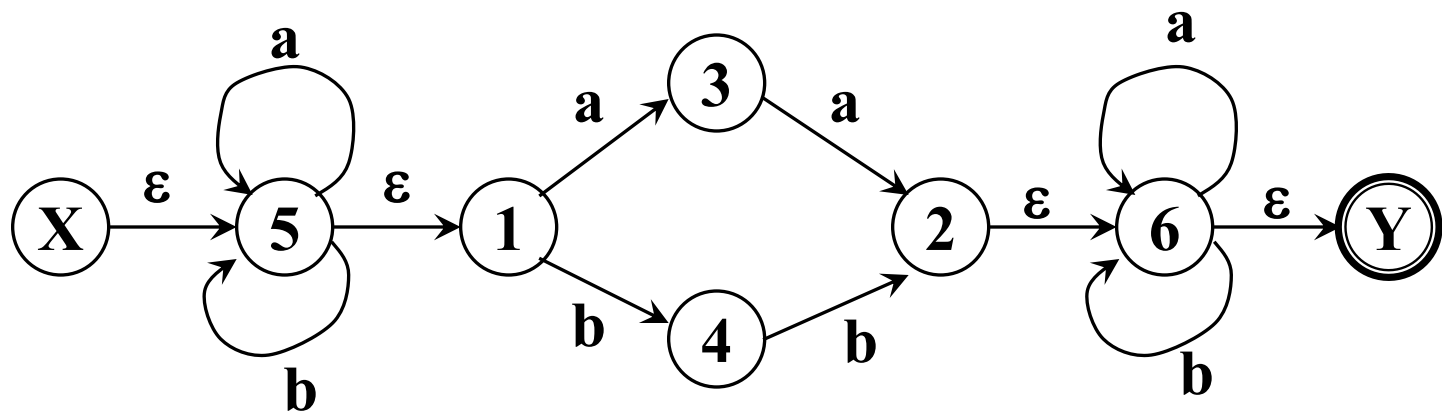


首先，置第1行第1列为 ϵ -closure($\{X\}$),
求出这一行的 I_a , I_b ;

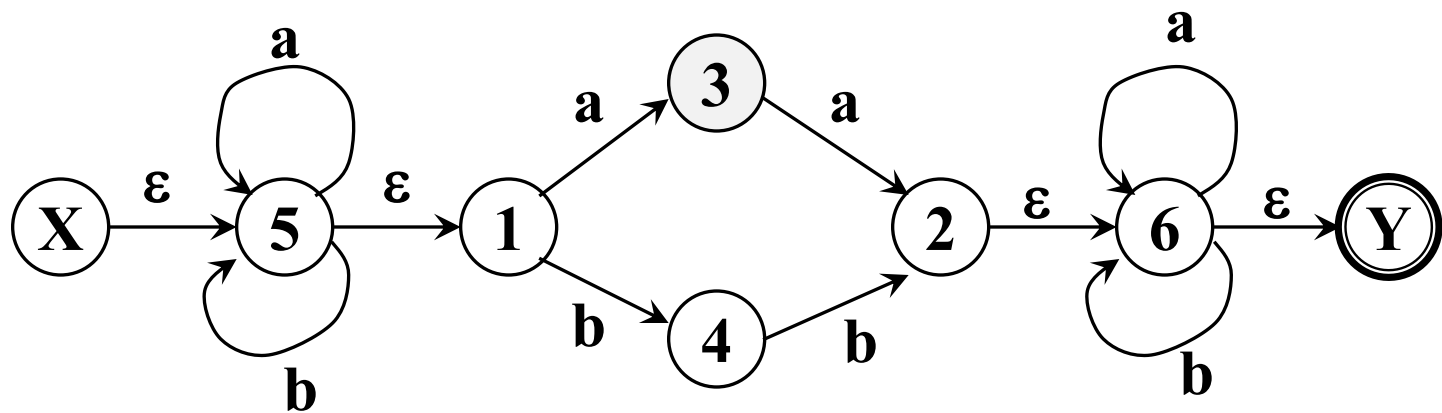
I	I_a	I_b
{X,5,1}	ϵ-closure(move(I, a))	ϵ-closure(move(I, b))



I	I_a	I_b
{X,5,1}	{5,3,1}	{5,4,1}



I	I_a	I_b
{X,5,1}	{5,3,1}	{5,4,1}
{5,3,1}		
{5,4,1}		



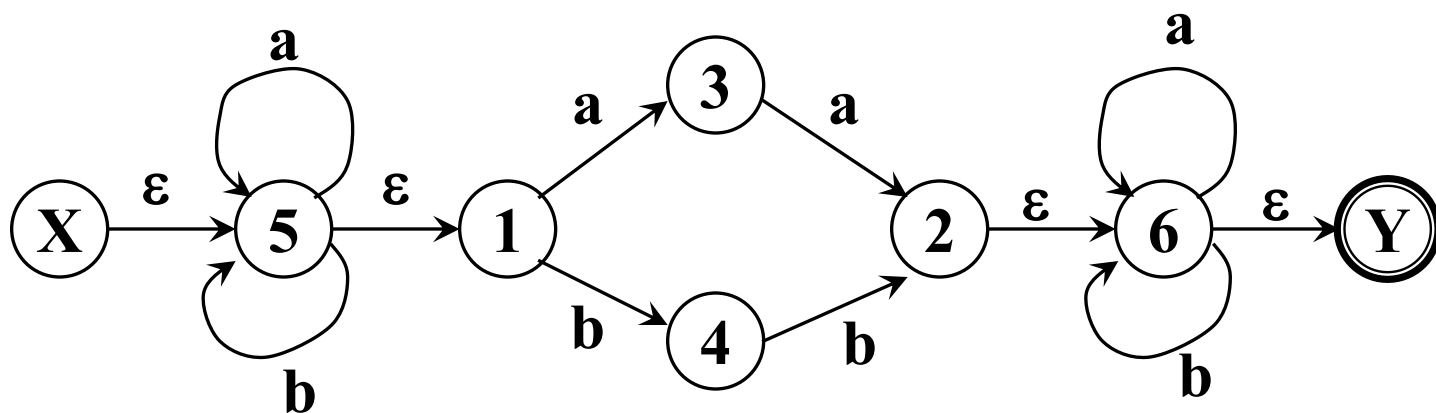
首先，置第1行第1列为 ε -closure($\{X\}$),
求出这一列的 I_a , I_b ;

然后，检查这两个 I_a , I_b ，看它们是否已在表中的第一列中出现，把未曾出现的填入后面的空行的第1列上，
求出每行第2, 3列上的集合...

重复上述过程，直至出现在第 $i+1$ ($i=1, 2, \dots, k$) 列上的所有状态子集均已在第1列上出现。

表3.3 对应于例3.3中正规式的状态转换矩阵

I	I _a	I _b
{X,5,1}	{5,3,1}	{5,4,1}
{5,3,1}	{5,2,3,1,6,Y}	{5,4,1}
{5,4,1}	{5,3,1}	{5,2,4,1,6,Y}
{5,2,3,1,6,Y}	{5,2,3,1,6,Y}	{5,4,6,1,Y}
{5,4,6,1,Y}	{5,3,6,1,Y}	{5,2,4,1,6,Y}
{5,2,4,1,6,Y}	{5,3,6,1,Y}	{5,2,4,1,6,Y}
{5,3,6,1,Y}	{5,2,3,1,6,Y}	{5,4,6,1,Y}



- 现在将构造出来的表视为状态转换表，把其中的每个状态子集视为新的状态。

- 现在将构造出来的表视为状态转换表，把其中的每个状态子集视为新的状态。
- 该表唯一地刻画了 **DFA M'** ，它的初态是该表首行首列的那个状态，终态是那些含有原终态 **Y** 的状态子集。

表3.3 对应于例3.3中正规式的状态转换矩阵

I	I_a	I_b
{X,5,1}	{5,3,1}	{5,4,1}
{5,3,1}	{5,2,3,1,6,Y}	{5,4,1}
{5,4,1}	{5,3,1}	{5,2,4,1,6,Y}
{5,2,3,1,6,Y}	{5,2,3,1,6,Y}	{5,4,6,1,Y}
{5,4,6,1,Y}	{5,3,6,1,Y}	{5,2,4,1,6,Y}
{5,2,4,1,6,Y}	{5,3,6,1,Y}	{5,2,4,1,6,Y}
{5,3,6,1,Y}	{5,2,3,1,6,Y}	{5,4,6,1,Y}

0={X, 5, 1}

2={5, 4, 1}

4={5,4,1,6,Y}

6={5,3,6,1,Y}

1={5, 3, 1}

3={5,2,3,1,6,Y}

5={5,2,4,1,6,Y}

表3.4 对表3.3中状态子集重新命名后的状态转换矩阵

I	a	b
0	1	2
1	3	2
2	1	5
3	3	4
4	6	5
5	6	5
6	3	4

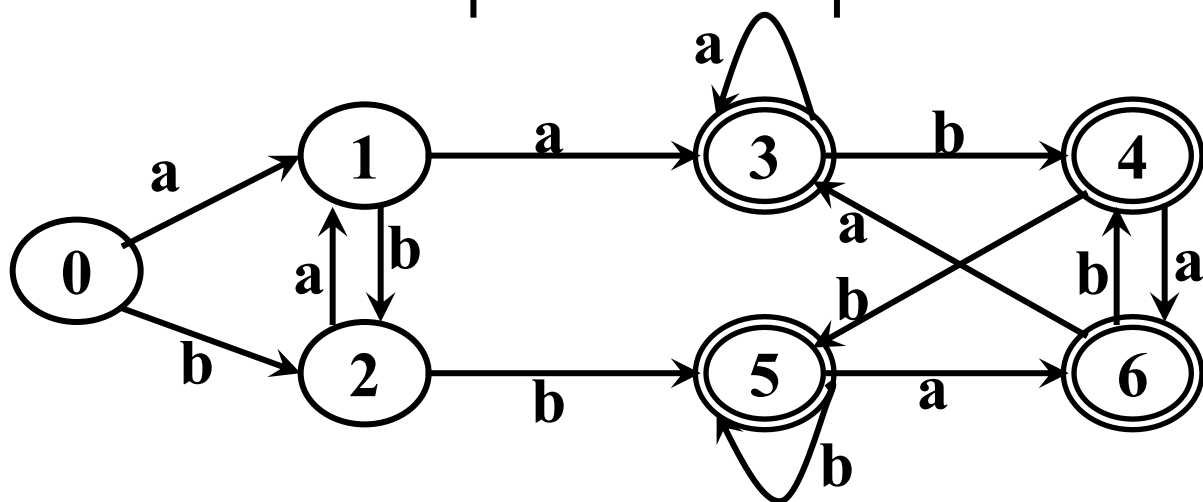


图3.8 未化简的DFA $(a|b)^*(aa|bb)(a|b)^*$

3.3.5 正规式与有限自动机的等价性

1. 对任何FA M ，都存在一个正规式 r ，使得 $L(r)=L(M)$ 。
2. 对任何正规式 r ，都存在一个FA M ，使得 $L(M)=L(r)$ 。

■ 证明1:

- 1 对于 Σ 上的NFA M , 构造 Σ 上的正规式 r , 使得 $L(r)=L(M)$ 。

■ 证明1:

将状态转换图的概念拓广，令每条弧可用正规式作标记。

(1) 在M的转换图上加进两个结点X和Y，从X用 ϵ 弧连接到M的所有初态结点，从M的所有终态结点用 ϵ 弧连接到Y，从而形成一个新的NFA，记为M'，它只有一个初态X和一个终态Y。显然 $L(M)=L(M')$ 。

(2) 反复使用图3.10的替换规则，逐步消去M'中的所有结点，直至只剩下X和Y为止。在消除结点的过程中，逐步用正规式来标记箭弧。最后，X到Y的弧上标记的正规式即为所构造的正规式r。显然 $L(r)=L(M)=L(M')$ 。



图3.10 替换规则

- 例：求出与如图所示的NFA M 所对应的正规式 r , 使 $L(r)=L(M)$ 。

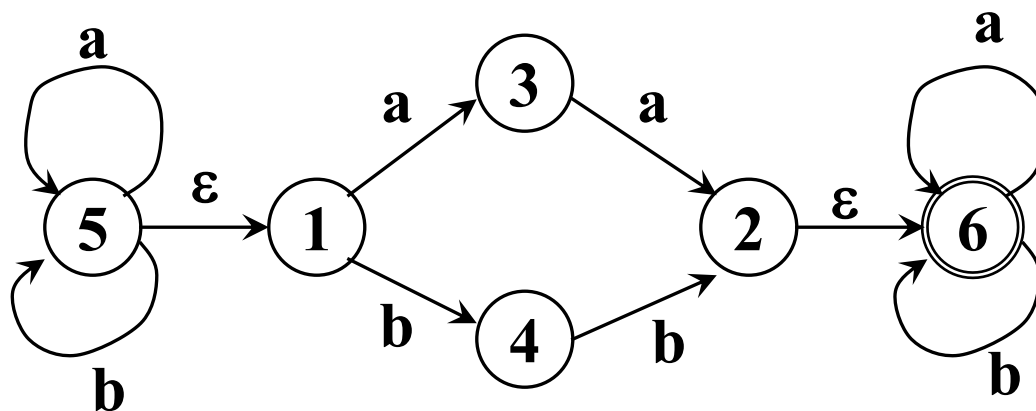


图3.6 非确定有限自动机

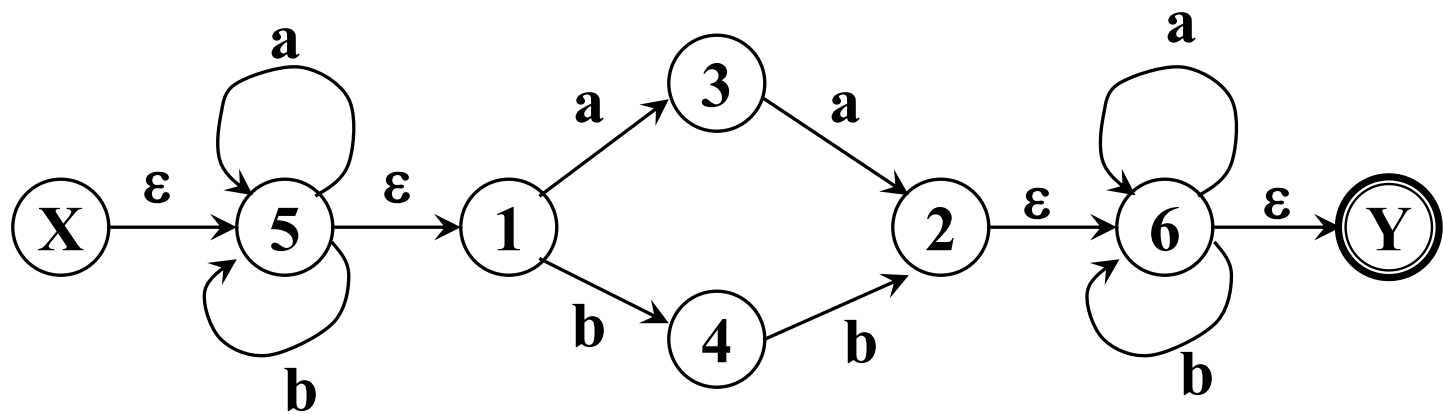


图3.6 非确定有限自动机

$(a|b)^*(aa|bb)(a|b)^*$

■ 证明2:

2 对于 Σ 上的正规式 r ，构造一个NFA M ，使 $L(M)=L(r)$ 。

■ 证明过程实质上是一个将正规表达式转换为有限自动机的算法。

基本思路

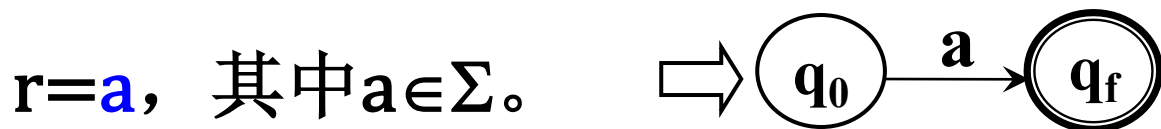
- 递归处理

- 每个子表达式，构造一个只有一个终态的NFA。

- 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

■ 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

(1) r 具有零个运算符，则



例: $r_3=0, r_1=1$

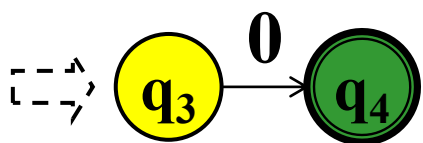


图 $r_3=0$ 对应的FA

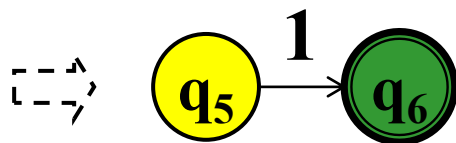


图 $r_1=1$ 对应的FA

- 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

(2) r 具有 k 个运算符

假设结论对于少于 $k(k \geq 1)$ 个运算符的正规式成立。

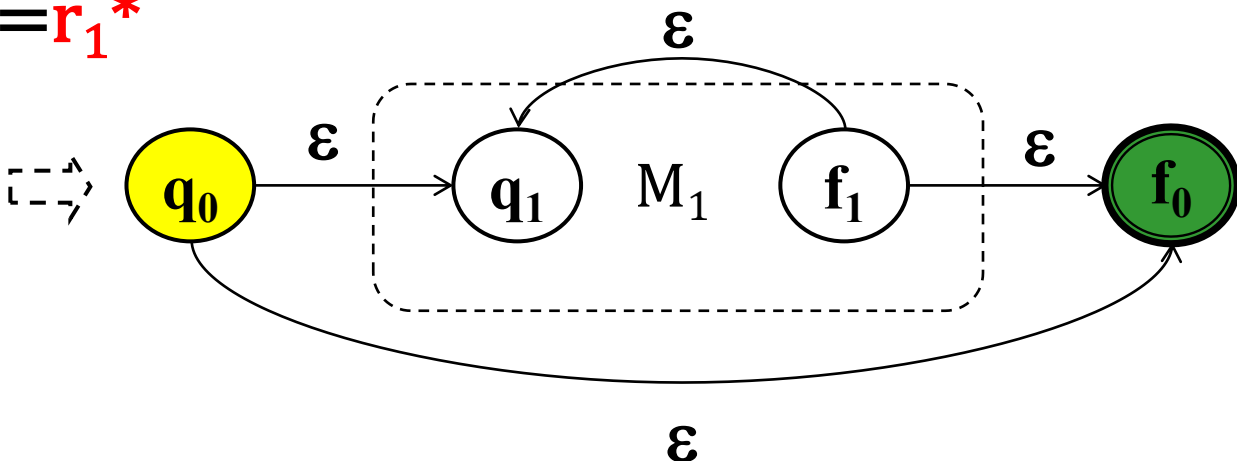
- $r=r_1|r_2$

- $r=r_1r_2$

- $r=r_1^*$

- 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

□ $r=r_1^*$



例: $r_2=1^*$

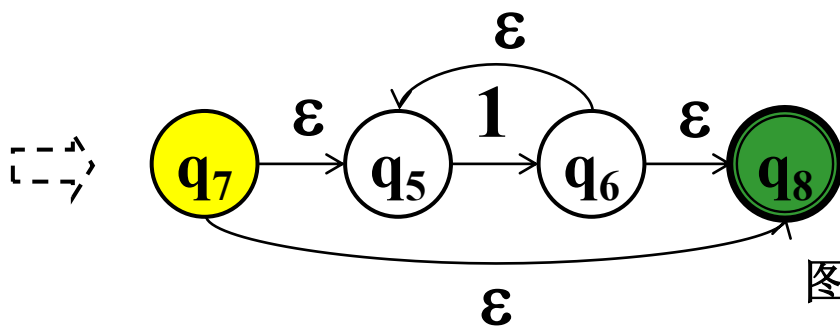
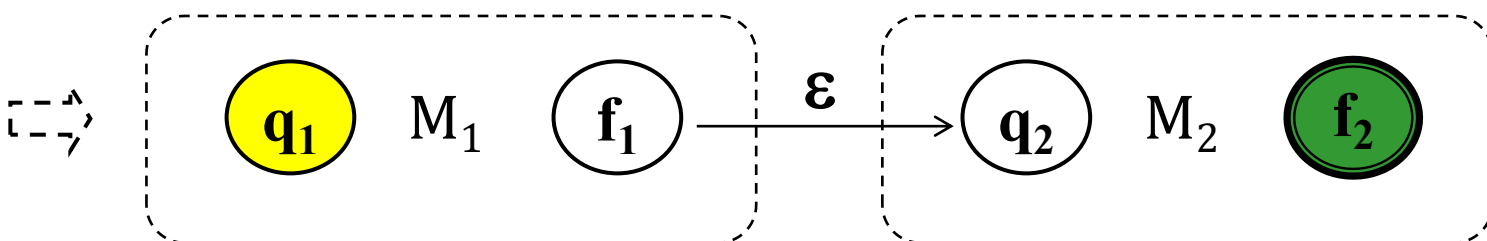


图 $r_2=1^*$ 对应的 FA

- 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

□ $r=r_1r_2$



例: $r_4=01^*$

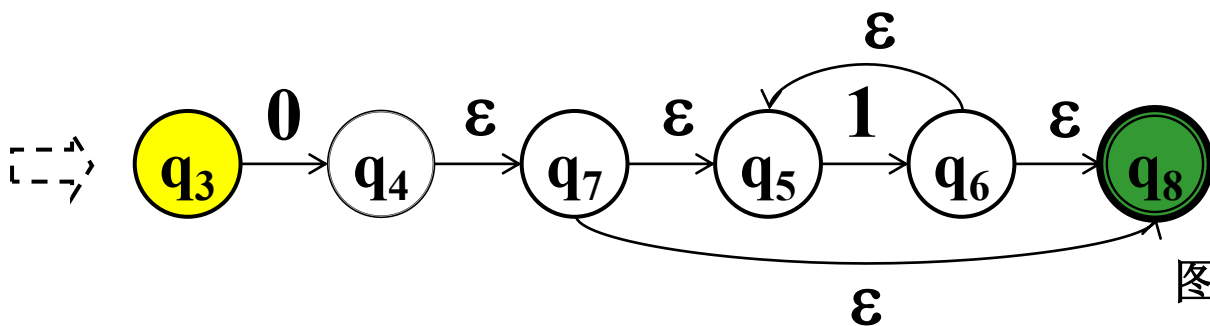
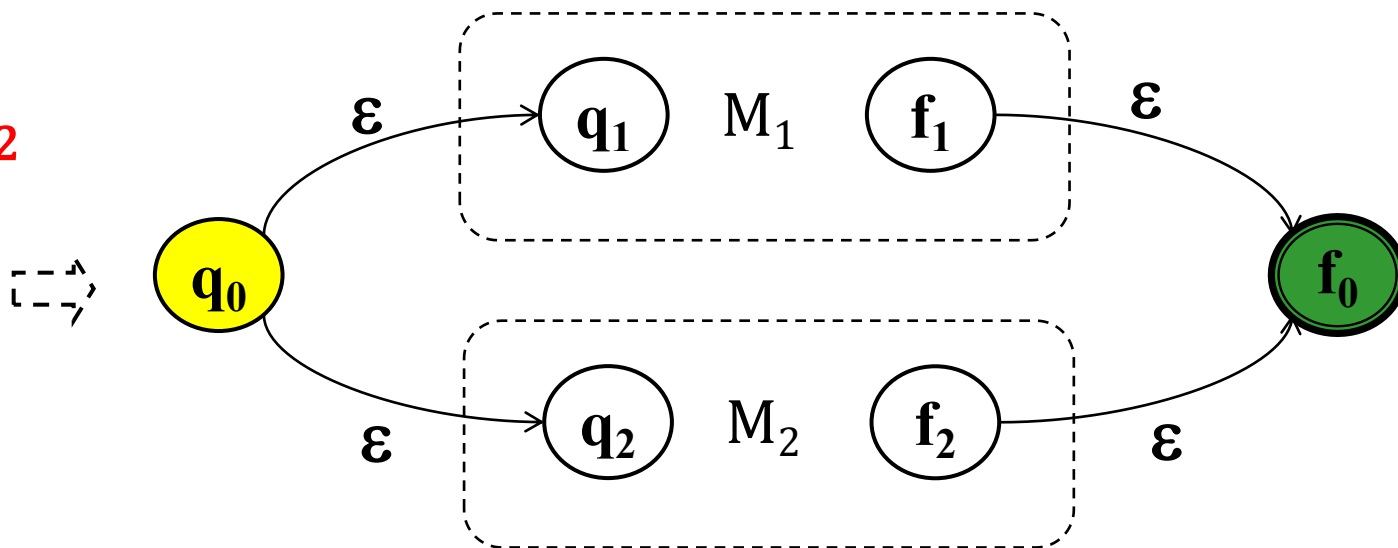


图 $r_4=01^*$ 对应的FA

■ 例3.5 求出与正规式 $r_6=01^*|1$ 等价的FA。

□ $r=r_1|r_2$



例: $r_6=01^*|1$

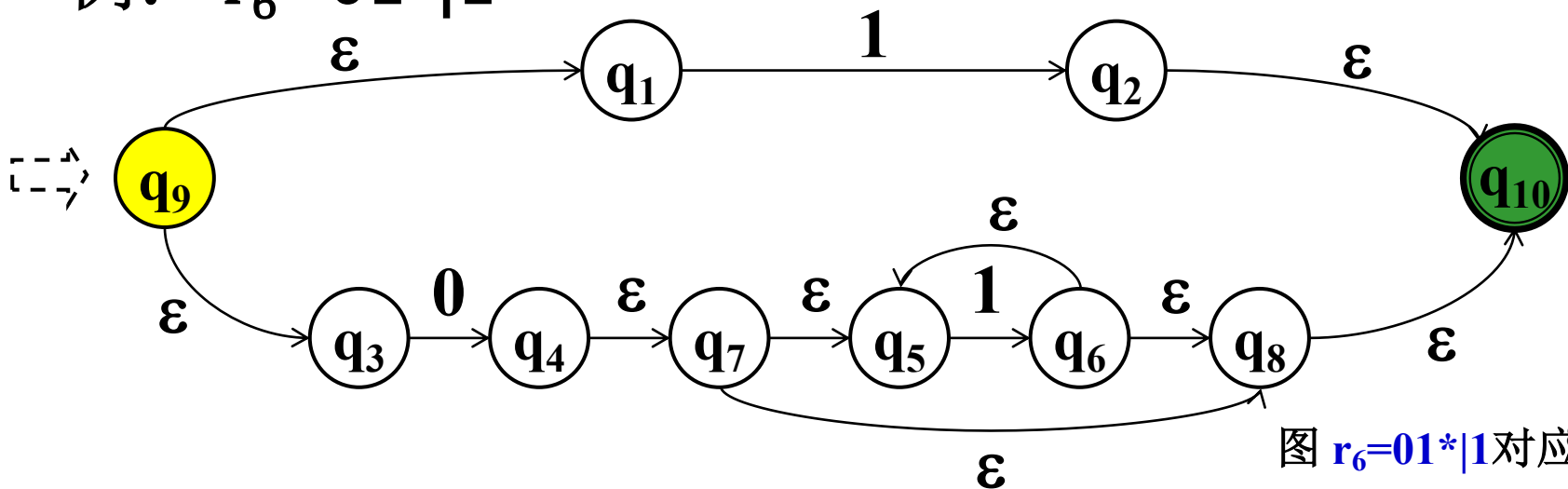


图 $r_6=01^*|1$ 对应的FA

3.3.6 确定有限自动机的化简

- DFA M 的化简: 寻找一个状态数比 M 少的 DFA M' , 使得 $L(M) = L(M')$ 。

3.3.6 确定有限自动机的化简

- 两个状态**等价**:

假设 s 和 t 是 M 的两个不同状态, 如果从状态 s 出发能读出某个字 w 而停止于终态, 那么同样, 从 t 出发也能读出同样的字 w 而停于终态; 反之亦然。

- 两个状态**可区别**:

如果DFA M 的两个状态 s 和 t 不等价, 则称这两个状态是可区别的。

例: **终态与非终态是可区别的。**

终态能读出空字 ϵ , 非终态则不能读出空字 ϵ 。

- DFA M 的状态最少化-分割法:
 - M 的状态集分割成不相交的子集
 - 任何不同的两子集中的状态都是可区别的
 - 同一子集中的任何两个状态都是等价的
 - 在每个子集中选出一个代表，同时消去其他等价状态。

- 具体做法: 对M的状态集S进行划分
 - 首先, 把S的终态与非终态分开, 分成两个子集, 形成基本划分 Π 。

例3.6 图3.8所示的DFA M的化简

非终态组: $\{0, 1, 2\}$

终态组: $\{3, 4, 5, 6\}$

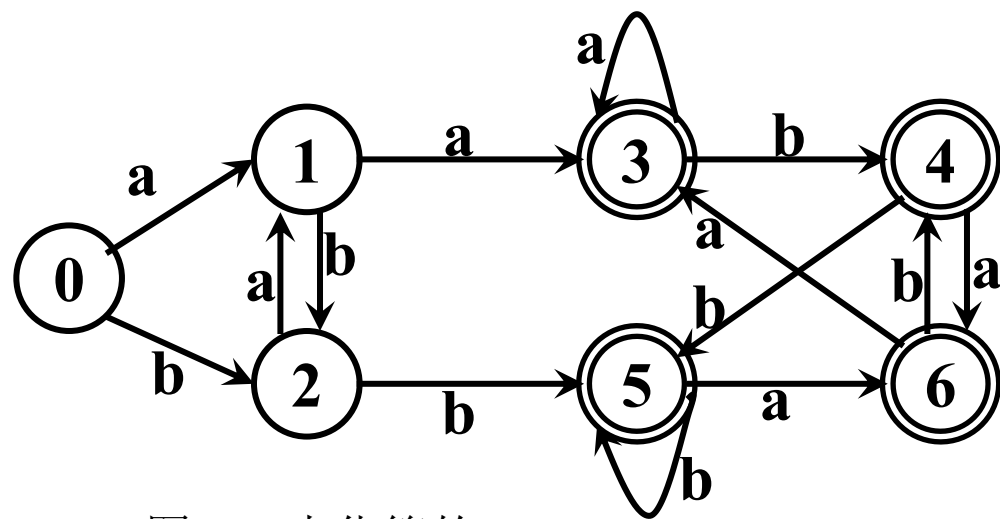


图3.8 未化简的DFA

- 具体做法: 对M的状态集进行划分
 - 首先, 把S的终态与非终态分开, 分成两个子集, 形成基本划分 Π 。

假定到某个时候, Π 已含m个子集, 记为 $\Pi=\{I^{(1)}, I^{(2)}, \dots, I^{(m)}\}$ 。

- 具体做法: 对M的状态集进行划分
 - 首先, 把S的终态与非终态分开, 分成两个子集, 形成基本划分 Π 。
- 假定到某个时候, Π 已含m个子集, 记为 $\Pi = \{I^{(1)}, I^{(2)}, \dots, I^{(m)}\}$ 。
- 检查 Π 中的每个子集 $I^{(i)}$ 看是否能进一步划分:
 - 对于某个 $I^{(i)}$, 令 $I^{(i)} = \{s_1, s_2, \dots, s_k\}$, 若存在一个输入字符a使得 $I_a^{(i)}$ 不全包含在现行 Π 的某个子集 $I^{(j)}$ 中, 则至少应把 $I^{(i)}$ 分为两个部分。

例3.6 图3.8所示的DFA M的化简 分析:

$$I^{(1)} = \{0, 1, 2\}$$

$$I^{(2)} = \{3, 4, 5, 6\}$$

$$I_a^{(1)} = \{0, 1, 2\}_a = \{1, 3\}$$

$I_a^{(1)}$ 既不包含在 $I^{(1)}$ 中, 也不包含在 $I^{(2)}$ 中。

$I^{(1)}$ 需要划分。

$$\{0\}_a = \{1\}, \{1\}_a = \{3\}, \{2\}_a = \{1\}$$

落入现行 Π 中 **2** 个不同子集。

将 $I^{(1)}$ 划分为 2 个不相交组。

$$I^{(11)} = \{0, 2\} \quad I^{(12)} = \{1\}$$

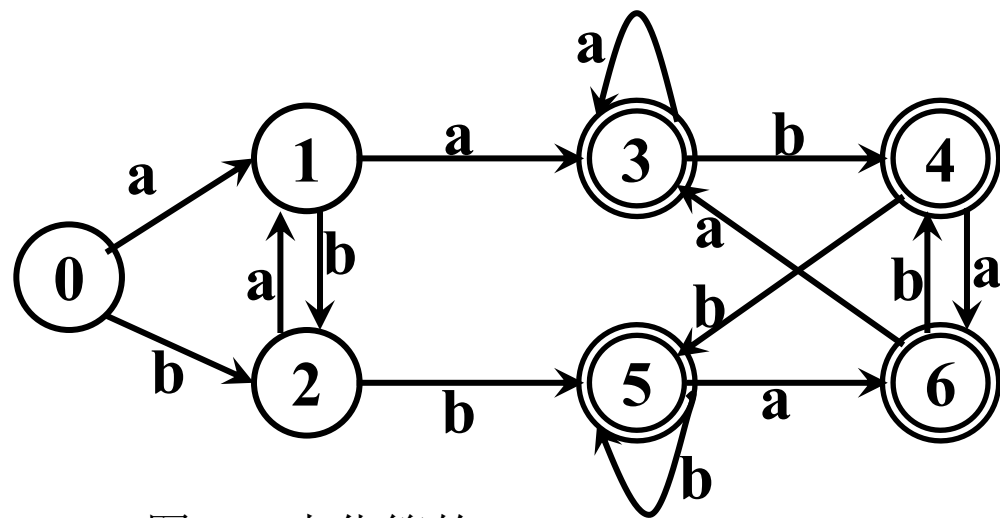


图3.8 未化简的DFA

例3.6 图3.8所示的DFA M的化简 分析:

$$I^{(1)} = \{0, 1, 2\}$$

$$I^{(1)} = \{3, 4, 5, 6\}$$

$$I^{(2)} = \{0, 2\}$$

$$I^{(3)} = \{1\}$$

$$I_a^{(1)} = \{0, 1, 2\}_a = \{1, 3\}$$

$I_a^{(1)}$ 既不包含在 $I^{(1)}$ 中, 也不包含在 $I^{(2)}$ 中。

$I^{(1)}$ 需要划分。

$$\{0\}_a = \{1\}, \{1\}_a = \{3\}, \{2\}_a = \{1\}$$

落入现行 Π 中 2 个不同子集。

将 $I^{(1)}$ 划分为 2 个不相交组。

$$I^{(11)} = \{0, 2\} \quad I^{(12)} = \{1\}$$

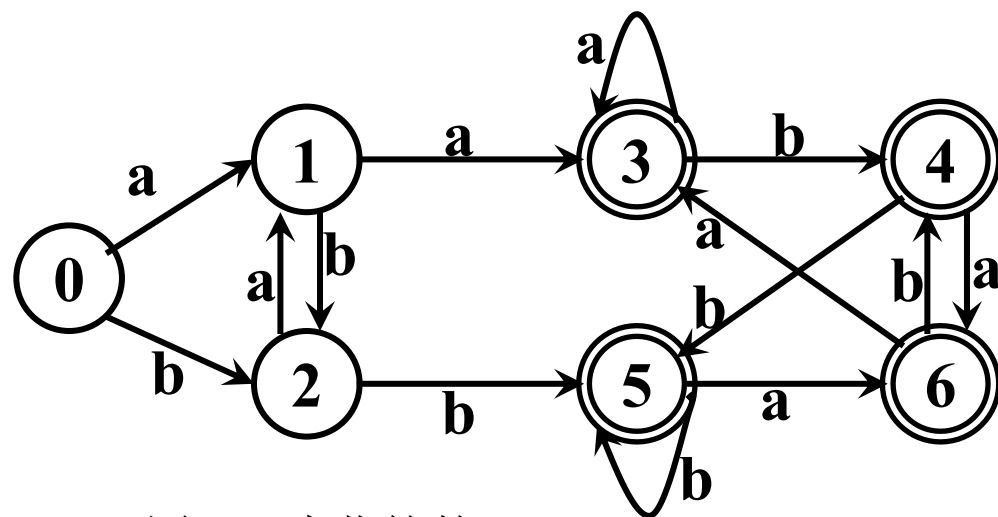


图3.8 未化简的DFA

例3.6 图3.8所示的DFA M的化简 分析:

$$I^{(1)} = \{3, 4, 5, 6\}$$

$$I^{(2)} = \{0, 2\}$$

$$I^{(3)} = \{1\}$$

$$I_a^{(1)} = \{3, 4, 5, 6\}_a = \{3, 6\}$$

$I_a^{(1)}$ 包含在 $I^{(1)}$ 中。

$$I_b^{(1)} = \{3, 4, 5, 6\}_b = \{4, 5\}$$

$I_b^{(1)}$ 包含在 $I^{(1)}$ 中。

$I^{(1)}$ 不需划分。

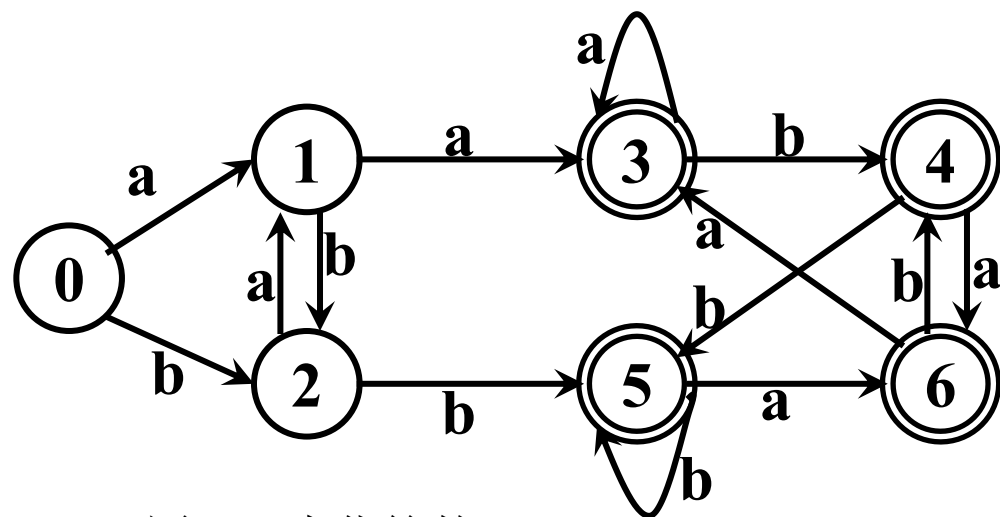


图3.8 未化简的DFA

例3.6 图3.8所示的DFA M的化简 分析:

$$I^{(1)} = \{3, 4, 5, 6\}$$

$$I^{(2)} = \{0, 2\}$$

$$I^{(3)} = \{1\}$$

$$I_a^{(2)} = \{0, 2\}_a = \{1\}$$

$$I_b^{(2)} = \{0, 2\}_b = \{2, 5\}$$

$I_b^{(2)}$ 不包含在任一组中。

$I^{(2)}$ 需要划分。

$$I^{(21)} = \{0\}, I^{(22)} = \{2\}.$$

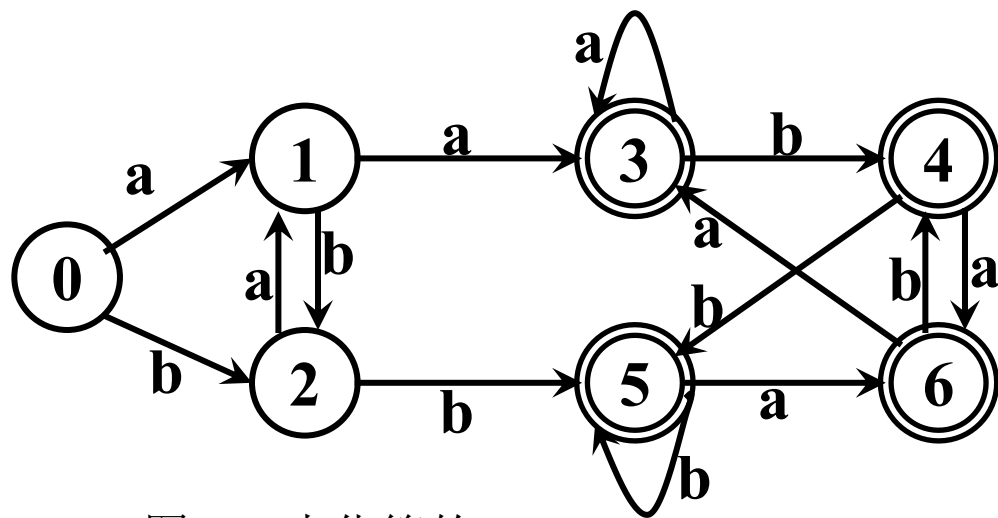


图3.8 未化简的DFA

例3.6 图3.8所示的DFA M的化简分析：

$$I^{(1)} = \{3, 4, 5, 6\}$$

$$\textcolor{green}{I^{(2)} = \{0, 2\}}$$

$$I^{(2)} = \{1\}$$

$$I^{(3)} = \{0\}$$

$$I^{(4)} = \{2\}$$

$$I_a^{(2)} = \{0, 2\}_a = \{1\}$$

$$I_b^{(2)} = \{0, 2\}_b = \{2, 5\}$$

$I_b^{(2)}$ 不包含在任一组中。

$I^{(2)}$ 需要划分。

$$\textcolor{blue}{I^{(21)} = \{0\}, I^{(22)} = \{2\}。}$$

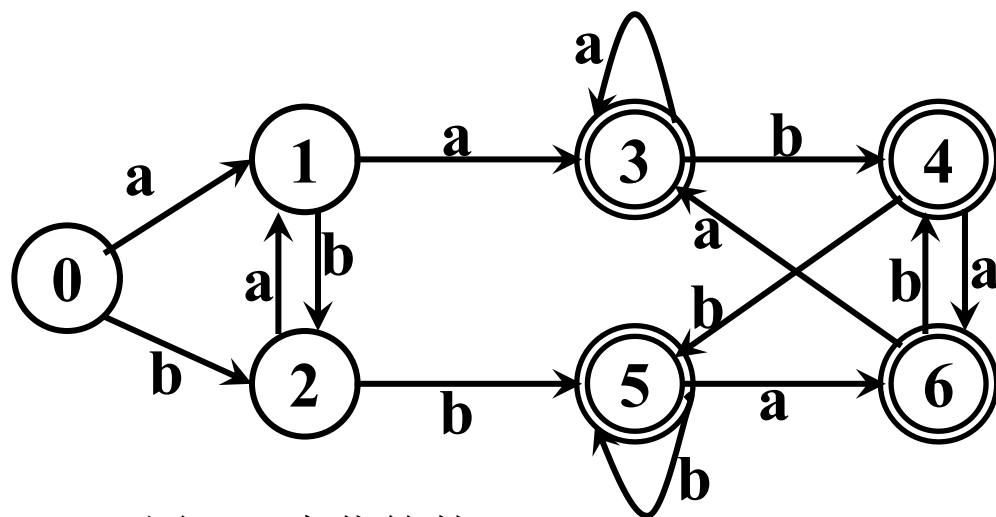


图3.8 未化简的DFA

- 具体做法: 对M的状态集进行划分

- 首先, 把S的终态与非终态分开, 分成两个子集, 形成基本划分 Π 。

假定到某个时候, Π 已含m个子集, 记为 $\Pi=\{I^{(1)}, I^{(2)}, \dots, I^{(m)}\}$ 。

- 检查 Π 中的每个子集 $I^{(i)}$ 看是否能进一步划分:

- 对于某个 $I^{(i)}$, 令 $I^{(i)}=\{s_1, s_2, \dots, s_k\}$, 若存在一个输入字符a使得 $I_a^{(i)}$ 不全包含在现行 Π 的某个子集 $I^{(j)}$ 中, 则至少应把 $I^{(i)}$ 分为两个部分。

- 重复上述过程, 直至划分中所含的子集数不再增长为止

- 经上述过程之后, 得到一个最后划分 Π 。

例3.6 图3.8所示的DFA M的化简

$I^{(1)} = \{3, 4, 5, 6\}$

$I^{(2)} = \{1\}$

$I^{(3)} = \{0\}$

$I^{(4)} = \{2\}$

表3.4 状态转换矩阵

I	a	b
0	1	2
1	3	2
2	1	5
3	3	4
4	6	5
5	6	5
6	3	4

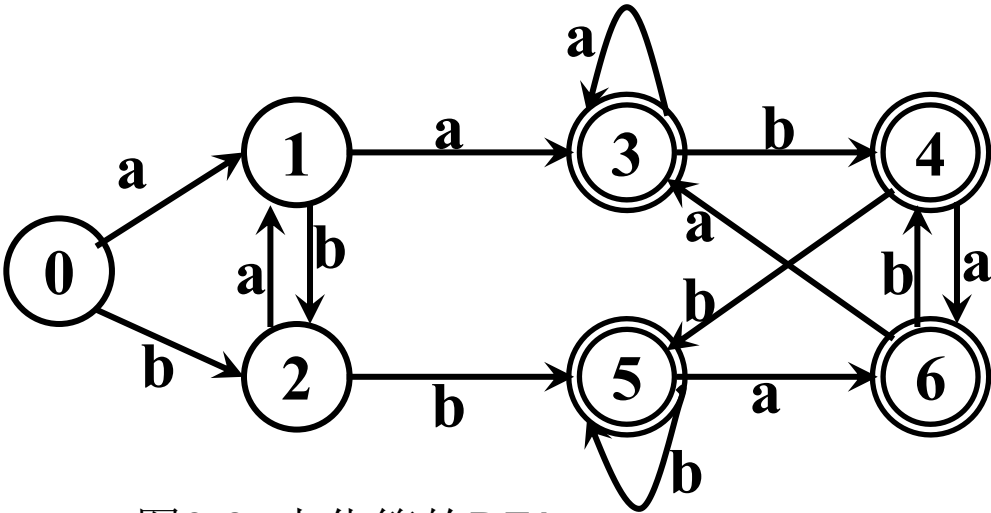


图3.8 未化简的DFA

- 具体做法: 对M的状态集进行划分
 - 对于这个 Π 中的每一个子集, 选取子集中的一个状态代表其它状态。例如, 假定 $I=\{s_1, s_2, \dots, s_k\}$ 是这样一个子集, 挑选 s_1 代表这个子集。
 - 在原来的自动机中, 凡导入到 $s_2, \dots, s_k$ 的弧都改成导入到 s_1 ;
 - 将 $s_2, \dots, s_k$ 从原来的状态集中删除;
 - 若I中含有原来的初态, 则 s_1 是新初态, 若I中含有原来的终态, 则 s_1 是新终态。

例3.6 图3.8所示的DFA M的化简

$I^{(1)} = \{3, \textcolor{teal}{4}, \textcolor{teal}{5}, \textcolor{teal}{6}\}$
 $I^{(2)} = \{1\}$
 $I^{(3)} = \{0\}$
 $I^{(4)} = \{2\}$

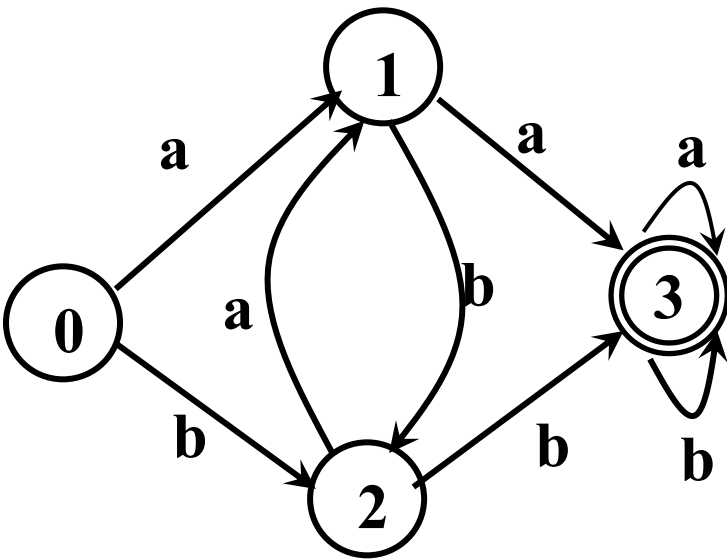


图3.5 状态转换图

表3.4 状态转换矩阵

I	a	b
0	1	2
1	3	2
2	1	5 3
3	3	4 3
4	6	5
5	6	5
6	3	4

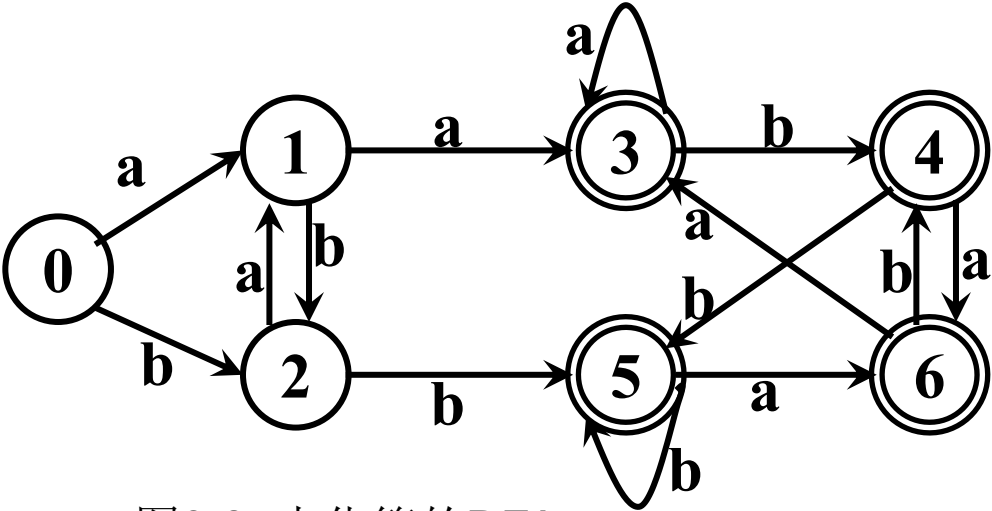
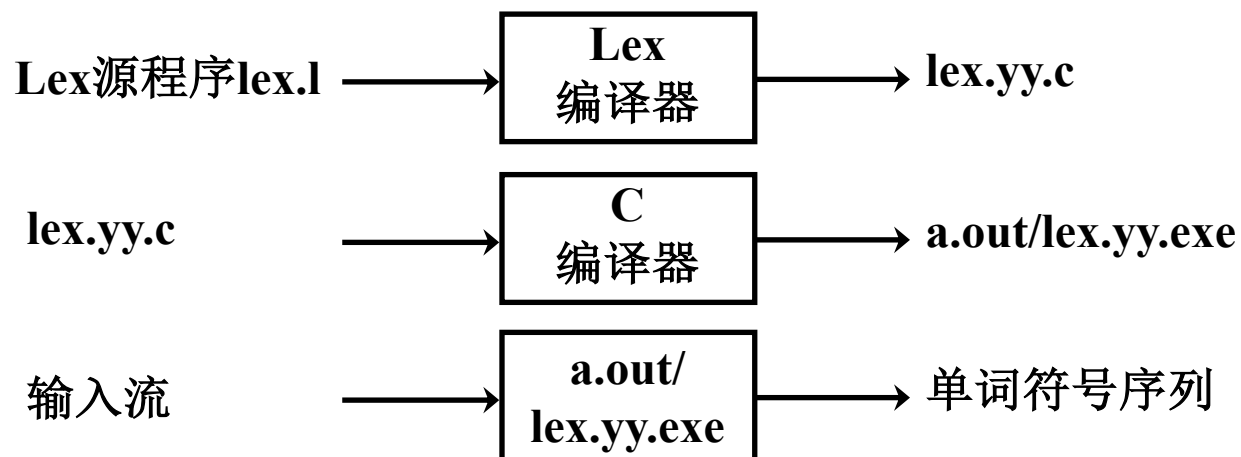


图3.8 未化简的DFA

3.4 词法分析器的自动产生

词法分析器生成工具Lex



3.4.1 语言LEX的一般描述

- LEX源程序的结构

声明部分

%%

识别规则

%%

辅助函数

3.4.1 语言LEX的一般描述

■ 1. 声明部分

```
%{  
    /*变量、符号常量等的声明;  
    直接复制到flex.yy.c中  
    */
```

```
%}  
正规定义式
```

3.4.1 语言LEX的一般描述

- 正规定义式

- 如果 Σ 是一个字母表, Σ 上的正规定义式:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_i \rightarrow r_i$$

其中, d_i 表示不同的名字;

每个 r_i 是 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的符号所构成的正规式。

- 对任何 r_i , 反复地将式中出现的名字代之以相应的正规式, 可以构成一个 Σ 上的正规表达式。

3.4.1 语言LEX的一般描述

- 正规定义式

- 例：标识符

$\text{letter_} \rightarrow [\text{A-Za-z_}]$

$\text{digit} \rightarrow [0-9]$

$\text{id} \rightarrow \text{letter_}(\text{letter_} | \text{digit})^*$

注：圆括号是用于分组的元符号，不代表圆括号本身。

3.4.1 语言LEX的一般描述

■ 2. 识别规则

P_1
 $\{A_1\}$

P_2
 $\{A_2\}$

... ...

P_m
 $\{A_m\}$

P_i : **正规式**, 称为词形, 是 $\Sigma \cup \{d_1, d_2, \dots, d_n\}$ 上的一个正规式, 最终转化为 Σ 上的正规式。

A_i : **一小段程序代码**。在识别出词形为 P_i 的单词之后, 词法分析器应采取的动作。

A_i 的基本组成成份: 返回 P_i 的种别编码和属性值。

return(code, value)

识别规则决定了词法分析器的功能, 分析器只能识别具有词形 $P_1, P_2, \dots, P_m$ 的单词符号。

3.4.1 语言LEX的一般描述

■ 3. 辅助函数

- 所有内容被直接复制到lex.yy.c中
- 位于识别规则之后，但可以在识别规则的动作定义中使用
- 如：InsertId()、InsertConst()

3.4.1 语言LEX的一般描述

- 词法分析器L如何工作？

L逐一扫描输入串的每个字符，

寻找一个最长的子串匹配 P_i ，

截取该子串放在TOKEN缓冲区；

L调用动作子程序 A_i ，

A_i 工作完后，L把单词符号交给语法分析程序。

当L重新被调用时，从输入串中继上次截出的位置之后识别下一个单词符号。

3.4.1 语言LEX的一般描述

- 匹配的问题:

- 零个: 找不到任何词形与输入串匹配, L报告输入串含有错误(如非法字符)。
- 多个: 一个最长子串可以匹配若干个不同的词形 P_i , 以LEX程序中**出现在最前面**的 P_i 为准。

3.4.3 LEX的实现

■ LEX程序的编译过程：

- 首先，对每条识别规则 P_i 构造一个 NFA M_i ；
- 然后，引进一个新初态 X ，通过 ε 弧，将这些自动机连接成一个新的NFA M ；
- 最后，将NFA M 改造成一个等价的DFA，化简DFA，生成状态转换表和控制程序。

